Traffic Oracle

Traffic speed forecasting on the METR-LA freeway network with graph attention networks, segment-level machine learning and leakage-safe hybrids — complete technical documentation

Team T042
Pranav Singh · Kunal · Himanshu · Kanchan · Vaibhav
Repository: github.com/Pranav-Singh-Devloper/traffic-oracle
Live demo and visualisation: traffic-oracle.streamlit.app
Document generated from the project's result files (results/metrics, configs, data manifest)

Contents

1. Executive summary **4**

2. Problem, hypothesis and scope **5**

Theory primer (read this first) **6**

Chronological splits . . . 6
Missing data . . . 6
Metrics . . . 7
XGBoost . . . 8
Random Forest . . . 8
Graph Attention Networks (GAT) . . . 9
Spatio-temporal modeling . . . 10
Diffusion convolution (DCRNN) . . . 10
Hybrid: residual learning with out-of-fold predictions . . . 11
Significance testing . . . 12
Simulated control: CTM, ALINEA . . . 12

3. Dataset **13**

3.1 Source and files . . . 13
3.2 Facts asserted on every load . . . 13
3.3 A loading problem and its fix (ADR-004) . . . 14
3.4 Missing data . . . 14

4. The road graph **15**

4.1 Construction (thresholded Gaussian kernel, as in DCRNN) . . . 15
4.2 Direction convention (ADR-002) . . . 15
4.3 Graph statistics . . . 15

5. Preprocessing pipeline (shared by all models) **16**

5.1 Global settings (configs/base.yaml) . . . 16
5.2 Chronological split . . . 16
5.3 Missing-data handling . . . 16
5.4 Scaling . . . 16
5.5 Input channels and windows . . . 16

6. Exploratory data analysis **18**

6.1 Missingness . . . 18
6.2 Daily profile and weekly pattern . . . 18
6.3 Speed distribution . . . 19
6.4 Autocorrelation . . . 19
6.5 Spatial dependence . . . 20
6.6 Propagation evidence (lagged cross-correlation) . . . 21
6.7 Where congestion happens . . . 22

6.8 Split comparability 22

7. Evaluation protocol (identical for all models) 23

7.1 Common prediction format 23
7.2 Metrics (masked, in mph) 23
7.3 Slices 23
7.4 Statistical significance 23
7.5 Automatic sanity checks 23

8. Baselines 24

9. Phase 1 — machine learning 25

9.1 Tabular features 25
9.2 Models 25
9.3 Hyperparameter tuning (Optuna, validation only) 25
9.4 Training budget (reduced for a laptop, ADR-006) 26
9.5 Phase 1 results 26
9.6 What drives the trees (gain and TreeSHAP) 27
9.7 Gap analysis: what do neighbour features add? 28
9.8 Side experiment: same-time-yesterday lag 29
9.9 Hardest sensors 29

10. Phase 2 — deep learning 30

10.1 The graph attention layer 30
10.2 STGAT architecture 30
10.3 GRU no-graph ablation 30
10.4 Training 30
10.5 Compute study and simplification (ADR-007) 31
10.6 Training outcomes 31
10.7 Results 31

11. Hybrid models 33

11.1 Why a residual hybrid 33
11.2 Leakage-safe out-of-fold base forecasts 33
11.3 Gating ensemble and simple average 33
11.4 Results 33

12. All results 36

12.1 Main table 36
12.2 Every horizon 36
12.3 Peak, off-peak and weekend 37
12.4 Congested vs free-flow 37
12.5 Uncertainty: day-block bootstrap 38
12.6 Significance (Diebold–Mariano) 38

13. Ablation studies **40**

14. Maps and interpretability **41**

14.1 Forecast maps . . . 41
14.2 Where the graph model helps . . . 41
14.3 Space-time diagram . . . 41
14.4 Attention (qualitative) . . . 42

15. Simulated ramp-metering experiment **43**

15.1 Why ramp metering . . . 43
15.2 Cell Transmission Model . . . 43
15.3 Demand (synthetic, shaped by METR-LA) . . . 43
15.4 Policies . . . 43
15.5 Results . . . 44

16. Engineering and reproducibility **45**

16.1 Repository layout . . . 45
16.2 Tests . . . 45
16.3 Decision records . . . 45
16.4 Hardware, runtimes and resource limits . . . 45
16.5 How to reproduce . . . 45

17. Limitations and honest findings **47**

17.1 Limitations . . . 47
17.2 Results that did not match the hypothesis . . . 47
17.3 Ethics and authorship . . . 47

18. Team T042 and contributions **47**

Appendix A. Glossary **47**

Appendix B. References **48**

1. Executive summary

This document describes the Traffic Oracle project end to end: the data, every preprocessing decision, the evaluation protocol, each model and its hyperparameters, all results with their uncertainty, the ablations, the maps, the simulated control study, the engineering set-up, and the limitations. Every number is read from the project's result files; nothing is typed by hand.

Live demo and visualisation: https://traffic-oracle.streamlit.app/ (interactive map, per-sensor forecasts, scoreboards, gap analysis, hybrids, ablations).

Task. Forecast freeway speed (mph) at 207 loop-detector sensors in Los Angeles, 5 to 60 minutes ahead (12 steps of 5 minutes), from the previous 60 minutes (12 steps). **Question.** Does information from neighbouring sensors (a road graph) improve on forecasts that use each sensor's own history, and if so, where?

Model	MAE @ 15 min	MAE @ 30 min	MAE @ 60 min
Persistence	3.494	4.198	5.368
Historical average	4.193	4.193	4.193
Seasonal naive	5.503	5.503	5.503
XGBoost per-segment	2.860	3.276	3.684
XGBoost global (pooled)	2.880	3.331	3.781
Random Forest (multi-output)	3.137	3.798	4.732
XGBoost spatial	2.820	3.258	3.730
GRU no-graph	2.892 ± 0.007	3.313 ± 0.013	3.761 ± 0.036
STGAT	2.901 ± 0.011	3.276 ± 0.021	3.666 ± 0.022
Hybrid: residual	2.820 ± 0.005	3.219 ± 0.002	3.647 ± 0.009
Hybrid: gating	2.836 ± 0.005	3.233 ± 0.012	3.607 ± 0.018
Hybrid: average	2.840 ± 0.005	3.246 ± 0.010	3.640 ± 0.018

Table 1. Headline test-set results (masked MAE in mph; ± is the standard deviation over 3 seeds for deep models).

- Every learned model beats persistence. Persistence MAE grows from 3.494 to 5.368 mph between 15 and 60 minutes.
- The best models are hybrids: the residual hybrid at 15 and 30 minutes (2.820 and 3.219 mph) and the gated hybrid at 60 minutes (3.607 mph).
- STGAT (graph attention) equals the no-graph GRU at 15 minutes (2.901 vs 2.892) and beats it clearly only at 60 minutes (3.666 vs 3.761).
- Hand-made neighbour features help XGBoost consistently, most at peaks and in congestion, but the gain does not grow with horizon.
- In reduced-budget ablations the road graph was not shown to beat a no-neighbour graph; node embeddings and calendar features mattered most.
- In an offline ramp-metering simulation, metering cut mainline congestion but forecasts did not reduce total time spent.

2. Problem, hypothesis and scope

Congestion on freeways is not local: a jam forms at a bottleneck and its tail moves upstream (a shockwave), so downstream sensors carry information about an upstream sensor's near future. A model that sees only a sensor's own history must infer this indirectly; a graph model can read neighbouring sensors directly.

Hypothesis. Graph-aware models improve accuracy over per-segment models, mostly at longer horizons and during peak periods, when propagation effects dominate.

Two phases. Phase 1 (machine learning): exploratory analysis, naive baselines, XGBoost per sensor and pooled, a multi-output Random Forest, and XGBoost with hand-engineered neighbour features (the bridge experiment). Phase 2 (deep learning): a spatio-temporal graph attention network (STGAT), a no-graph GRU ablation, a residual hybrid and a gated hybrid that combine XGBoost with the graph model, ablations, maps, and an optional simulated ramp-metering experiment.

> Control is simulation only. Every control result in this document comes from an offline simulation on a synthetic network. It demonstrates how forecasts could inform a control policy; no live or real-world traffic control is performed or claimed.

Theory primer (read this first)

The concepts every team member must be able to explain, in the order they appear in the pipeline. Each part ends with likely viva questions.

Chronological splits

Owner: Kanchan

Idea

Traffic speed is strongly autocorrelated and we slide windows with stride 1, so neighboring samples share almost all of their inputs and targets. A random train/test split puts near-copies of test samples in training: the model is rewarded for memorising the immediate past, and the measured error is far lower than what it would achieve on genuinely new days.

Our protocol (strict)

- Split the **timestamp axis**: first 70% train, next 10% validation, last 20% test (`chronological_split`).
- Build windows **inside** each split. Window input and target never cross a boundary, so `max(train target_ts) < min(val input_ts)` and `max(val target_ts) < min(test input_ts)` (tested).
- Everything learned from data (scaler, time-of-day means for imputation, historical-average table, target encodings, hyperparameters) uses train (or val for tuning) only.

DCRNN protocol (literature-comparable)

DCRNN builds all windows first, then splits the *samples* 70/10/20 (23974/3425/6850 here). Windows at a boundary overlap across splits (up to 23 steps), a small leak. We use it only to compare with published numbers and label it as such.

Equations

Purged boundary: with history H and horizon F, windows whose span `[t-H+1, t+F]` is wholly inside one split are the only valid ones, so each split loses `H+F-1` anchors.

Likely viva questions

- *Why not k-fold cross validation?* It shuffles time; use forward-chaining (expanding window) if CV is needed (we do this for OOF predictions in the hybrid).
- *Does forward-fill leak?* No: it only looks backwards in time.
- *Why report the DCRNN protocol at all?* Comparability with published MAE tables; it is labeled.
- *What distribution shift exists between our splits?* See `docs/eda_summary.md`: the test period has a higher outage rate.
- *Why can validation still overfit?* We pick hyperparameters and early-stop on it; test is touched once.

Missing data

Owner: Kanchan

Zero is not "no traffic"

In METR-LA a speed of 0 almost always means the loop detector did not report. Treating it as a real speed teaches a model that sensors randomly crash to 0 mph.

Masking

Targets: never imputed; loss and metrics sum only over observed entries: `L = Σ m·ℓ(y,ŷ) / Σ m`. Gradients come only from real observations, and normalising by Σm keeps the scale independent of the batch's missing rate. The masked loss is an unbiased estimate of the loss over observed entries if missingness is independent of the target given the inputs (an assumption we cannot fully verify: outages cluster in time).

Inputs

Option (a, default): forward-fill up to 6 steps (30 min) using only the past, then train-only time-of-day means, plus an `is_missing` channel. Option (b, DCRNN): leave as 0 in raw space; we set it to the train mean in scaled space (ADR-003) and rely on the `is_missing` channel. Ablation A9 compares them.

Facts in this dataset

See `docs/eda_summary.md`: ~8% missing, whole-network outages exist, and the test period has more outages than train.

Likely viva questions

- *Why not interpolate the targets?* It would invent data and reward smoothness.
- *Why a missing flag when imputing?* The model should know the value is not real.
- *Where could masking bias results?* If outages correlate with congestion.

Metrics

Owner: Himanshu

All metrics are computed in **mph**, on **observed** targets only (mask = speed > 0).

- **MAE** = `Σ m|y−ŷ| / Σ m`: average error in mph; robust, same units as speed.
- **RMSE** = `sqrt(Σ m(y−ŷ)² / Σ m)`: penalises large errors more; larger than MAE whenever errors vary. A big RMSE/MAE gap signals occasional big misses (e.g. missing the onset of congestion).
- **MAPE** = `100·Σ m'|y−ŷ|/y / Σ m'`, `m' = m ∧ (y ≥ 1 mph)`: relative error; unstable at low speeds because the denominator is small, and asymmetric (over-prediction costs more than under-prediction). Use it as secondary.

Why per horizon

Error grows with horizon; a single averaged number hides where models differ. Headline horizons are 15/30/60 min (steps 3/6/12); all 12 are in `all_metrics_long.csv`.

Why slices

Congestion is rare overall but frequent at peaks; averaged metrics are dominated by easy free-flow samples. We also report peak, off-peak weekday, weekend, congested (<40 mph) and free-flow.

Edge cases

An all-masked slice returns NaN, never 0. NaN predictions (untrained horizons) are removed from the mask.

Likely viva questions

- *Why masked metrics?* Zero readings are outages, counting them would reward/penalise models for nothing.
- *MAE vs RMSE choice?* Report both; MAE is our training/early-stopping metric.
- *Why not MAPE as headline?* Instability at low speed.

- *Why is historical average flat across horizons?* It ignores the current state, so its error does not depend on horizon.

XGBoost

Owner: Kunal

Model

Additive trees: $\hat{y}_i = \Sigma_\{k=1\}^K f_k(x_i)$, each f_k a regression tree with leaf weights w.

Objective and second-order approximation

$L = \Sigma_i\ l(y_i,\ \hat{y}_i) + \Sigma_k\ \Omega(f_k)$, $\Omega(f) = \gamma T + \frac{1}{2}\lambda\|w\|^2$ (T leaves). At step t, expand the loss to second order around the current prediction: $L^(t) \approx \Sigma_i\ [g_i\ f_t(x_i) + \frac{1}{2}\ h_i\ f_t(x_i)^2] + \Omega(f_t)$, with $g_i = \partial l/\partial\hat{y}$, $h_i = \partial^2 l/\partial\hat{y}^2$. For a leaf containing index set I the optimal weight is $w* = -G/(H+\lambda)$ ($G = \Sigma g$, $H = \Sigma h$) with score $-\frac{1}{2}\ G^2/(H+\lambda)$.

Split gain

$Gain = \frac{1}{2}[\ G_L^2/(H_L+\lambda) + G_R^2/(H_R+\lambda) - (G_L+G_R)^2/(H_L+H_R+\lambda)\ ] - \gamma$ A split is made only if gain > 0: γ is a complexity price per leaf, λ shrinks leaf weights.

Practical details used here

- `tree_method="hist"` (histogram binning, fast), early stopping on validation MAE.
- Objective is chosen by Optuna from squared error / absolute error / pseudo-Huber; MAE is the reported metric, and absolute error targets the conditional median.
- **Per-segment vs global:** per-segment trains one model per sensor (207 x horizons, each ~24k rows, no sharing). Global pools sensors (~5M rows) with sensor id and static features, borrowing strength across sensors but still without neighbor information.

Likely viva questions

- *Why second order?* Newton step gives faster, more stable fitting and a closed-form leaf weight.
- *What do λ and γ do?* λ shrinks leaf weights; γ prunes splits with small gain.
- *Difference between gain importance and SHAP?* Gain is split-based and biased toward high-cardinality/ early splits; SHAP attributes each prediction consistently (we use TreeSHAP via `pred_contribs`).
- *Why can XGBoost not model propagation?* A row only contains that sensor's own history; neighbor information must be hand-engineered (XGB-spatial).
- *Why do trees handle 'speed==0'-like issues badly?* We impute inputs and add missing flags; targets are masked.

Random Forest

Owner: Vaibhav

Idea

Average many decorrelated, deep trees. Each tree is fit on a bootstrap sample (`max_samples`) and each split considers a random subset of features (`max_features`).

Variance reduction

For B trees with variance σ^2 and pairwise correlation ρ: $Var(mean) = \rho\sigma^2 + (1-\rho)\sigma^2/B$. Bagging shrinks the second term; feature subsampling lowers ρ, which is the floor of the variance. Bias stays about that of

one deep tree.

Native multi-output

`RandomForestRegressor` fits one tree per bootstrap sample for all outputs at once: a split minimises the **sum of squared errors over all 12 horizons**, so a leaf stores a 12-vector. Predictions across horizons are coherent and one model replaces 12, but the split criterion is dominated by long horizons (larger errors).

Choices here

- Pooled over sensors, rows with any missing horizon target dropped (share logged), training rows subsampled (see ADR-006) for memory/time.
- Squared-error criterion (MAE is evaluated, so RF is not tuned for the metric directly).

Likely viva questions

- *Why doesn't adding trees overfit?* Variance falls monotonically with B (bias is unchanged).
- *Why max_features < all?* Decorrelates trees (lowers ρ).
- *Why does RF extrapolate poorly?* Leaves output training-set averages, so predictions stay in the observed range.
- *Multi-output pros/cons?* Coherent horizons and speed vs. no horizon-specific tuning.

Graph Attention Networks (GAT)

Owner: Pranav Singh

Implemented in `src/traffic/models/layers/gat_dense.py` and tested against PyG (`tests/test_gat_layer.py`).

Equations (one head)

- `h'_i = W h_i`
- `e_ij = LeakyReLU(a^⊤ [h'_i ‖ h'_j])` for `j ∈ N(i) ∪ {i}`, $-\infty$ otherwise
- α`_ij = softmax_j(e_ij)` (masked softmax over neighbors)
- `h_i^out =` Σ`_j` α`_ij h'_j (+ b)`, then ELU outside the layer. K heads are concatenated (averaged in an output layer).

Splitting `a = [a_dst; a_src]` gives `e_ij = a_dst^⊤ h'_i + a_src^⊤ h'_j`, which is what we compute (cheap: two dot products, then a broadcast sum).

Why dense and masked

N = 207, so an [N x N] attention matrix per head is small; masking with the adjacency (non-edges to $-\infty$) gives exactly the sparse softmax and batches trivially. Cost $O(B{\cdot}K{\cdot}N^2{\cdot}d)$ vs sparse $O(B{\cdot}K{\cdot}|E|{\cdot}d)$.

Direction convention

`adj[i, j] > 0` means node i aggregates from node j (ADR-002). PyG's `edge_index` is (source j, target i) so the test builds it with `adj.nonzero().flip(1).T`.

GCN vs GAT vs GATv2

- **GCN** weights neighbors by fixed normalised adjacency (`attention="mean"` ablation): same weights for every input.
- **GAT** learns input-dependent weights, but its attention is *static* in ranking: with `a^⊤[Wh_i‖Wh_j]` the ordering of neighbors j is the same for every query node i (the score splits into a function of i plus a function of j, so softmax over j ignores i).

- **GATv2** moves the nonlinearity inside: `a`$^\top$ `LeakyReLU(W[h_i‖h_j])` ⇒ dynamic attention (`gatv2` ablation). We compute it in checkpointed chunks because the [B,N,N,K,D] tensor is large.

Edge bias

`e_ij +=` $\beta\cdot$`W_ij` adds a learnable distance prior (ablation A12).

Likely viva questions

- *What does the mask do?* Restricts the softmax to neighbors so non-neighbors get exactly 0 attention.
- *Why LeakyReLU?* Keeps gradients for negative scores; ELU is applied to the output.
- *Why multiple heads?* Different heads can specialise (e.g. upstream vs downstream), and averaging stabilises training.
- *What is "static attention"?* See above; GATv2 fixes it.
- *Complexity?* See above.
- *Why add self-loops?* So a node can keep its own information; also avoids all $-\infty$ rows.

Spatio-temporal modeling

Owner: Pranav Singh

Shockwaves

Congestion starts at a bottleneck and the jam's tail moves upstream at roughly 10–15 mph (our CTM backward wave speed w ≈ 12 mph). A sensor upstream of a bottleneck will slow down minutes after downstream sensors do. A per-segment model sees only its own past and learns this only indirectly, while a graph model can read the downstream state directly.

Receptive field

Spatial reach = L (GAT layers) hops; temporal reach = H input steps (TCN receptive field grows with dilation: kernel 3, dilations 1, 2, 4...). A shockwave at speed w covers distance w·Δt in time Δt, so the graph depth needed for horizon h grows with h. Our evidence: correlation decays with graph distance (EDA fig. 6), but directional lags are only weakly asymmetric (EDA fig. 7).

Over-smoothing

Repeated neighborhood averaging makes node representations similar. Residual connections, LayerNorm and few layers (L = 2 default, tested at 1 and 3, ablation A6) limit this.

Non-autoregressive decoding

All 12 horizons are predicted jointly: no error accumulation from feeding predictions back, at the cost of no explicit horizon-to-horizon dependency.

Likely viva questions

- *Why a gated causal temporal conv?* Parallel over time (faster than RNN), causal padding avoids using future steps (tested).
- *Why a node embedding?* Gives each sensor an identity (local behaviour); ablation A7.
- *What would more layers do?* Larger receptive field but over-smoothing and cost.

Diffusion convolution (DCRNN)

Owner: Vaibhav

Optional model (`dcrnn_lite`); the main project uses attention instead.

Idea

Traffic on a directed road graph behaves like a random walk. With transition matrices $D_O^{-1} W$ (forward) and $D_I^{-1} W^\top$ (backward), a K-step bidirectional diffusion filter is $Z = \sum_{k=0}^{K-1} (\theta_{k,1} (D_O^{-1}W)^k + \theta_{k,2} (D_I^{-1}W^\top)^k) X$. Inside a GRU cell, the matrix multiplications of the gates are replaced by diffusion convolutions, giving the DCRNN encoder–decoder.

Why both directions

Information flows downstream (vehicles move) but congestion waves travel upstream, so the model needs both W and $W^\top$.

Contrast with GAT

Diffusion weights are fixed by distance; GAT learns weights from node features. Diffusion reaches K hops through matrix powers, while stacked GAT layers reach L hops.

Likely viva questions

- *Why is the graph directed?* Road distance from i to j differs from j to i.
- *What does K control?* Receptive field in hops.
- *Why a random walk?* Gives a normalised, stochastic operator whose powers stay bounded.

Hybrid: residual learning with out-of-fold predictions

Owner: Kunal

Rationale

XGBoost captures local autoregression and calendar effects cheaply. Spatial propagation is its blind spot (Phase 1 gap analysis). We train a graph model on the **residual** $r = y - \hat{y}_XGB$, giving it $\hat{y}$_XGB as an input, so it concentrates on what XGBoost misses. The last decoder layer is zero-initialised, so training starts exactly at the XGBoost forecast.

Why out-of-fold

XGBoost's predictions on its own training rows are overfit: residuals there are unrealistically small, so the graph model would learn to correct almost nothing and then face larger residuals on val/test (train/test mismatch). Out-of-fold (OOF) predictions have realistic error magnitudes.

Forward-chaining OOF (time series)

Split train into 5 chronological blocks. For block $k \geq 2$, train on blocks $< k$ (leave a gap of $H+F-1$ anchors) and predict block k. Block 1 has no OOF forecast and is dropped from residual training. Random K-fold would leak the future into the models. Caveat: early-block models are trained on less data, so their OOF errors are a bit larger than the final model's val/test errors (conservative direction).

Stacking view

This is stacking with a time-aware base-learner protocol: level-0 = XGBoost, level-1 = graph residual learner trained on OOF level-0 outputs.

Gating ensemble

$\hat{y}$ = w·$\hat{y}$_STGAT + (1−w)·$\hat{y}$_XGB, w = σ(MLP(horizon one-hot, peak flag, tod, missing rate)), trained on the first 80% of val and early-stopped on the last 20% (val is then no longer pure for model selection; documented limitation).

Likely viva questions

- *Why not use in-sample predictions?* See above.
- *Why forward-chaining?* Respect time order; no future in training.
- *What if the hybrid does not beat STGAT?* Report it: STGAT may already capture the local structure.
- *Why zero-init?* Start at a strong baseline; training only adds improvements.

Significance testing

Owner: Himanshu

Diebold–Mariano (DM)

Compare two forecasts' loss series d_t = L(e1_t) − L(e2_t) (absolute error averaged over sensors, per horizon). Under H0, E[d_t] = 0. Statistic DM = d̄ / sqrt(V̄/T) where V̄ is a long-run variance.

Why HAC

Test anchors overlap (stride 1) and errors are persistent, so d_t is autocorrelated. A naive variance is too small, giving false significance. We use a Newey–West (Bartlett) variance with L = max(h−1, ⌈T^{1/3}⌉) lags, the Harvey–Leybourne–Newbold small-sample correction and a t(T−1) reference. The classic rule uses h−1 lags (forecast errors at horizon h are MA(h−1)); we take the larger lag for robustness.

Block bootstrap

Resample whole days (blocks of 288 steps) with replacement, recompute MAE; the 2.5/97.5 percentiles give a 95% CI. For paired differences the same days are resampled for both models, so day-level difficulty cancels.

Caveats

Multiple comparisons (many horizons/slices) inflate false positives; we report effect sizes with CIs. DL models use 3 seeds: DM is run per seed pair and we report the median statistic and the largest p-value.

Likely viva questions

- *Why not a paired t-test?* Autocorrelated losses.
- *What does a negative DM statistic mean here?* Model 1 has the lower loss.
- *Why blocks of one day?* Daily seasonality is the dominant dependence.

Simulated control: CTM, ALINEA

Owner: Himanshu

Disclaimer: offline simulation on a synthetic network; no live or real-world control is performed or claimed.

Cell Transmission Model

Discretises a freeway into cells of length L; vehicles per cell n_i. Per step: sending S_i = min(v·dt/L · n_i, Q_i·dt), receiving R_i = min(Q_i·dt, w·dt/L·(N_jam − n_i)), flow min(S_i, R_{i+1}). Triangular fundamental diagram: free-flow speed v, capacity q_max at critical density ρ_c = q_max/v, backward

wave speed `w = q_max / (ρ_jam − ρ_c)`. CFL: `v·dt ≤ L`. Merges share receiving capacity with a mainline priority.

ALINEA

`r(k) = r(k−1) + K_R (o* − o(k))`: an integral controller steering downstream occupancy to the critical value. Reactive: it responds after density rises.

Predictive ALINEA

Replace o(k) by `max(o_measured, o_from_forecast)`, where the forecast is the STGAT +15 min speed converted to density on the congested FD branch. In free flow, speed is uninformative about density on a triangular FD, so the measured value is used. Oracle version uses true future speed (upper bound).

Honest limits

METR-LA has speed only, so demand is a synthetic profile shaped by the observed congestion index. The simulator is open loop: its congestion does not influence the forecasts. Real ramp metering also involves safety, queue-override and driver behaviour not modelled. METR-LA freeways have no signals, hence ramp metering rather than intersection control.

Likely viva questions

- *Why CTM?* Simple, conservative, reproduces queue growth and backward waves.
- *Why ALINEA?* Standard, robust feedback law with one gain.
- *Why can speed not give density in free flow?* All sub-critical densities have the same speed on a triangular FD.

3. Dataset

3.1 Source and files

METR-LA is the benchmark released with DCRNN (Li et al., ICLR 2018), from loop detectors on Los Angeles County freeways (origin cited by DCRNN as Jagadish et al., CACM 2014). The speed file comes from the DCRNN Google Drive folder; the sensor-graph files come from the `data/sensor_graph/` folder of the DCRNN GitHub repository. `scripts/download_data.py` fetches all five files (with 3 retries and back-off), verifies them, and writes `data/raw/MANIFEST.json` with sizes and SHA-256 hashes.

File	Bytes	SHA-256 (first 16 hex)	Content
metr-la.h5	57,038,056	64784b76d6fb8ec9	speed table (pandas HDF5, key 'df')
adj_mx.pkl	680,459	a35687c6e15aa228	(sensor_ids, id_to_index, 207×207 adjacency)
distances_la_2012.csv	6,393,348	a576a2a3e28dbb95	directed road distances (from, to, cost)
graph_sensor_ids.txt	1,448	3ba026caa2e6263a	207 sensor ids, comma-separated
graph_sensor_locations.csv	6,341	eb8ea96e07358b45	sensor id, latitude, longitude

Table 2. Raw files, as verified by the download script.

3.2 Facts asserted on every load

- Shape 34,272 timestamps × 207 sensors; 34,272 = 119 days × 288 five-minute steps.
- First timestamp 2012-03-01 00:00:00, last 2012-06-27 23:55:00; the index is strictly regular at 5 minutes (no gaps, no duplicates).
- Values are speeds in mph, non-negative, no NaN; maximum observed speed 70.0 mph.
- **Speed only.** There is no flow and no occupancy (those exist in PeMS datasets).
- **A reading of 0 mph means the sensor did not report** (outage), not stopped traffic. Zeros are masked in losses and metrics and never used as targets.

- Sensor ids from the HDF5 columns, the pickle, the id list and the location file are normalised to strings and must agree; all arrays use the order of `adj_mx.pkl`.

3.3 A loading problem and its fix (ADR-004)

With pandas 3.1 and PyTables 3.11, `pd.read_hdf` fails on this 2019 file with "TypeError: a bytes-like object is required, not 'str"' (old attribute encoding). The loader therefore falls back to reading the plain fixed-format layout directly with PyTables: `axis0` (column ids), `axis1` (int64 nanosecond timestamps) and `block0_values` (float64 34,272×207). The data are identical and all integrity checks pass.

3.4 Missing data

Statistic	Value
Readings equal to 0 (missing), whole series	8.11%
Per-sensor missing rate: median	6.97%
Per-sensor missing rate: maximum	20.10%
Sensors that are never missing	0
Timestamps with $\geq$ 90% of sensors missing (network-wide outages)	2,148
Days containing at least one network-wide outage	61
Longest single-sensor outage	1,683 steps ($\approx$ 5.8 days)
Missing rate: train / validation / test	7.1% / 7.0% / 12.1%

Table 3. Missingness in METR-LA.

Missingness is partly clustered in time (whole-network outages), not only per sensor, and the **test period has a much higher outage rate** than training. Metrics are masked, so this does not bias them, but test results are computed over fewer observations.

4. The road graph

4.1 Construction (thresholded Gaussian kernel, as in DCRNN)

The 11,753 rows of `distances_la_2012.csv` that connect two of the 207 sensors are placed in a 207×207 matrix of directed road distances (metres; missing pairs = ∞). With σ the standard deviation of the finite entries (σ = 2,584.5 m) and $\kappa = 0.1$:

$$W_ij = \exp(\ -(\text{dist}(v_i, v_j) \ / \ \sigma)^2\ ) \text{ if this value} \geq \kappa, \text{ else } 0$$

The rebuilt matrix is compared with the published `adj_mx.pkl`: maximum absolute difference 1.02e-07, with 0 entries differing by more than 1e-6 (float32 rounding only). The pickle is used as the canonical matrix; the rebuild is the proof of understanding.

4.2 Direction convention (ADR-002)

`W[i, j]` is built from the road distance from sensor i to sensor j. Throughout the code, `A[i, j] > 0` means **node i aggregates information from node j**. With `forward = W`, node i receives from sensors it drives towards (downstream, where shockwaves start); with `backward` = $W^\top$, from sensors that drive into it (upstream). The default STGAT graph is `bidirectional = W + W`$^\top$. A unit test checks the convention on a hand-built three-sensor chain a→b→c.

4.3 Graph statistics

Statistic	Value
Nodes	207
Directed edges (off-diagonal non-zeros)	1,515
Density	0.0355
Self-loops in the matrix	207
Mean in/out degree	7.32
Non-zero off-diagonal weights: min / mean	0.100 / 0.401
Weakly connected components (largest)	2 (206 nodes; one isolated sensor)
Strongly connected components (largest)	13 (195 nodes)
Diameter of the largest strongly connected component	17 hops

Table 4. Statistics of the canonical directed graph (NetworkX).

Variant	Definition	Off-diagonal edges	Max neighbours incl. self	Mean neighbours incl. self
forward	W (downstream senders)	1,515	19	8.3
backward	$W^\top$ (upstream senders)	1,515	18	8.3
bidirectional	$W + W^\top$ (default)	2,626	26	13.7
identity	no neighbours	0	1	1.0
random	same edge count, random positions (seeded)	1,515	15	8.3
knn_geo	8 nearest by haversine, Gaussian weights	1,656	9	9.0

Table 5. Graph variants used in the models and ablations.

5. Preprocessing pipeline (shared by all models)

5.1 Global settings (configs/base.yaml)

Setting	Value	Meaning
seed	42	global random seed
interval_minutes	5	time step
history_steps (H)	12	60 minutes of input for every model
horizon_steps (F)	12	predict 5 … 60 minutes ahead
report_horizons	[3, 6, 12]	15, 30, 60 minutes in headline tables
split	70 / 10 / 20 %	chronological, by timestamp
peak windows	06:00–10:00 and 15:00–19:00, weekdays	judged on the target timestamp
congestion threshold	40 mph	congested vs free-flow slice
input_imputation	ffill_tod_mean	forward fill $\leq$ 6 steps, then train time-of-day mean
scaler	global	one masked z-score for all sensors
split_protocol	strict	windows built inside each split
use_daily_lag	False	same-time-yesterday lag off in the main comparison
resources	6 CPU workers, 4 torch threads	keeps the laptop responsive

Table 6. Global configuration.

5.2 Chronological split

Split	First timestamp	Last timestamp	Steps	Windows (strict)
train	2012-03-01 00:00:00	2012-05-23 07:05:00	23,990	23,967
val	2012-05-23 07:10:00	2012-06-04 04:40:00	3,427	3,404
test	2012-06-04 04:45:00	2012-06-27 23:55:00	6,855	6,832

Table 7. Strict chronological split. Each split loses $H + F - 1 = 23$ anchors so no window crosses a boundary.

A unit test asserts that the last training target precedes the first validation input, and the last validation target precedes the first test input. The literature-comparable DCRNN protocol (all windows first, then split samples) gives 34,249 samples split 23,974 / 3,425 / 6,850; its boundary windows overlap across splits, so it is only used for comparison and is tested but not used for the reported results.

5.3 Missing-data handling

- **Mask:** `mask = speed > 0`, shape [T, N].
- **Targets:** never imputed; masked in every loss and metric.
- **Inputs (default):** forward-fill outages for up to 6 steps (30 minutes, past only, so no leakage), then fill with the training-period mean of that sensor at that time-of-day slot (288 slots × 207 sensors); fall back to the sensor's training mean, then the global training mean.
- **Missing flag:** a binary `is_missing` input channel tells the model which values are not real.
- **Option (b), ablation A9:** keep the zeros; after scaling they are set to 0 in scaled space (the training mean) instead of the misleading $(0 - \text{mean})/\text{std} \approx -4.6$ (ADR-003).

5.4 Scaling

A masked z-score fitted on training timestamps and observed values only: mean = 58.584 mph, standard deviation = 12.823 mph. Models train in scaled space; every metric is computed in mph after the inverse transform. A test asserts that changing validation/test values never changes the scaler or the time-of-day means.

5.5 Input channels and windows

#	Channel	Definition
0	speed	imputed speed, z-scored
1	is_missing	1 if the original reading was 0
2	tod_sin	$\sin(2\pi \cdot \text{minute_of_day} / 1440)$
3	tod_cos	$\cos(2\pi \cdot \text{minute_of_day} / 1440)$
4	dow_sin	$\sin(2\pi \cdot \text{day_of_week} / 7)$
5	dow_cos	$\cos(2\pi \cdot \text{day_of_week} / 7)$
6	is_weekend	Saturday or Sunday
7	is_holiday	US federal holiday; in range only Memorial Day 2012-05-28 (validation period)

Table 8. The eight input channels (C = 8). Cyclic encoding keeps 23:55 and 00:00 close.

$$X[t] = \text{channels}[t - 11 \ldots t] \to [12, 207, 8] \quad Y[t] = \text{speed}[t + 1 \ldots t + 12] \to [12, 207], \text{ with mask alongside}$$

Windows store only their anchor index and slice the shared array on demand (the fully materialised training tensor would be ≈ 1.9 GB). Deep models use a vectorised batch iterator that indexes the array directly; validation and test batches keep chronological order.

6. Exploratory data analysis

All ten figures are produced by `scripts/run_eda.py`; numbers are in `results/metrics/eda_stats.json` and the auto-generated `docs/eda_summary.md`. Figures 5–7 use the training period only because they inform modelling choices.

6.1 Missingness

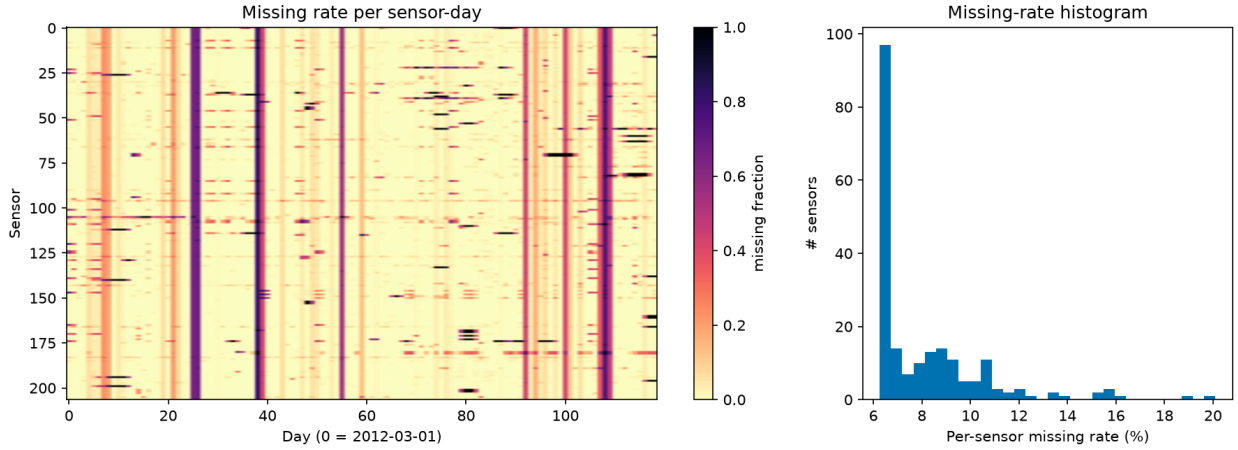


Figure 1. Missing rate per sensor-day (left) and per-sensor missing-rate histogram (right).

6.2 Daily profile and weekly pattern

Weekday mean speed bottoms out at 45.3 mph around 17:30; weekends only drop to 56.8 mph (around 13:30). Across the week, slot means range from 42.5 to 65.7 mph, with congestion confined to weekday rush hours.

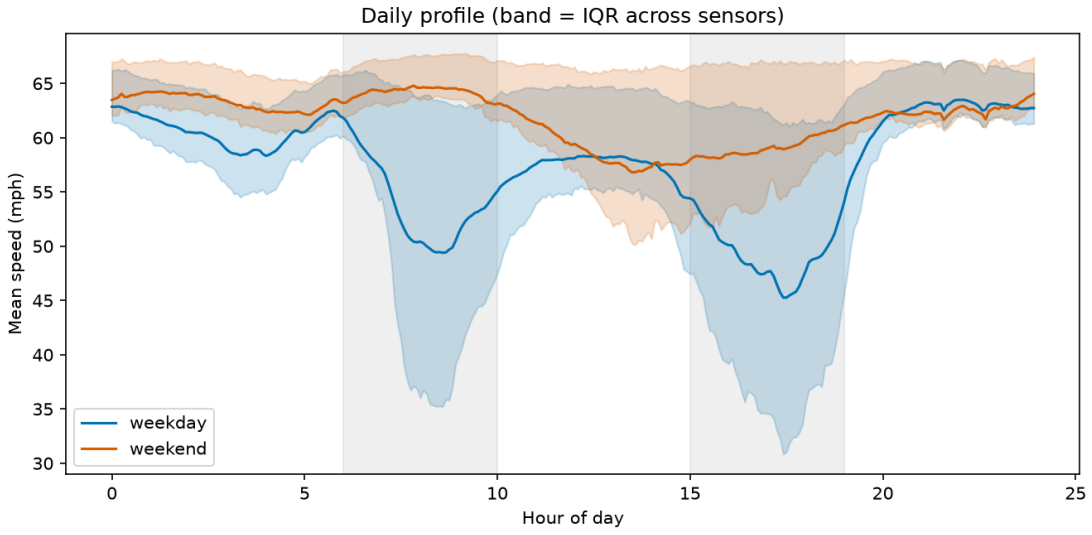


Figure 2. Mean speed by time of day, weekday vs weekend (band: interquartile range across sensors; shaded: peak windows).

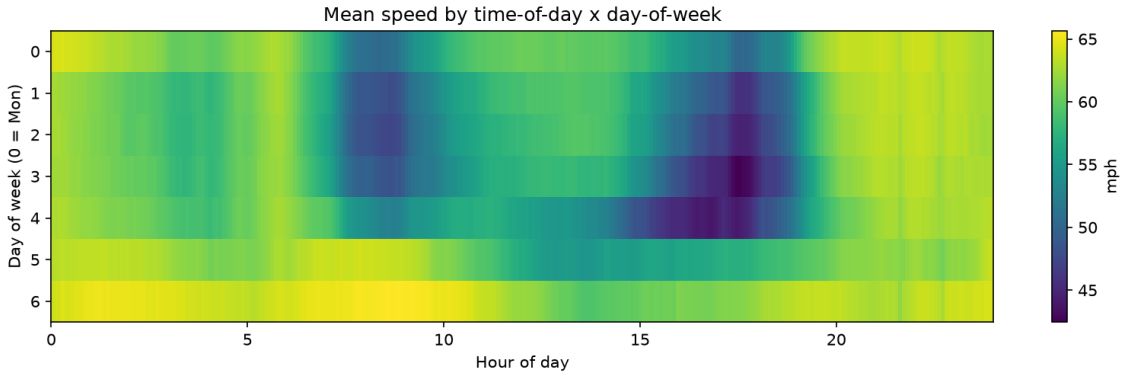


Figure 3. Mean speed by time of day × day of week.

6.3 Speed distribution

Overall mean 58.46 mph. 10.9% of observations are below 40 mph overall, 26.9% during weekday peaks and 5.8% off-peak (mean 51.5 mph at peak vs 60.6 mph off-peak). Congestion is the minority class and dominates errors.

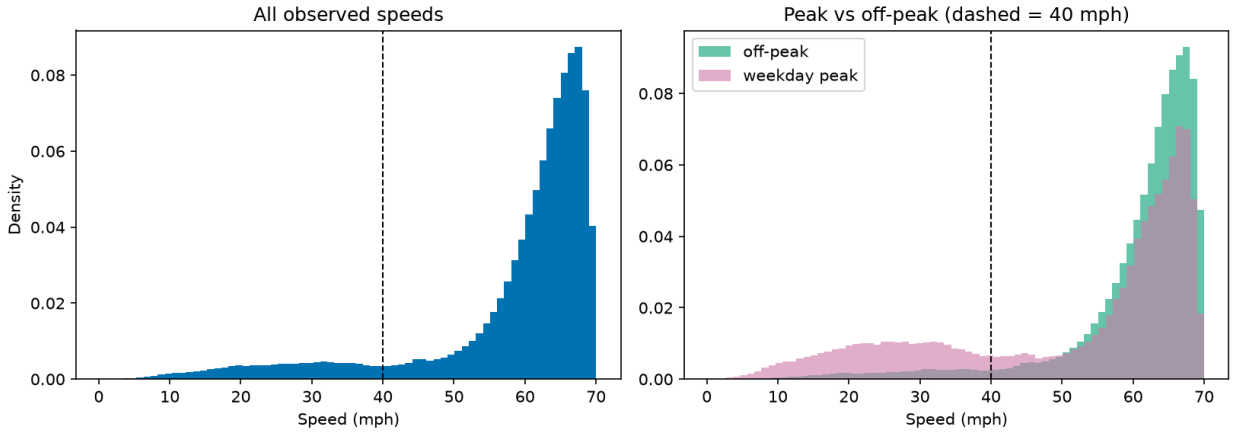


Figure 4. Speed distribution overall and peak vs off-peak (dashed: 40 mph).

6.4 Autocorrelation

Three representative sensors (lowest, median and highest variance among sensors with $< 10\%$ missing) show autocorrelation at the daily lag of 288 steps of 0.034, 0.400, 0.455 respectively: daily seasonality exists but its strength varies by sensor. Short-lag autocorrelation is very strong, which justifies a 12-step history and makes nearby windows dependent (a reason for chronological splits and autocorrelation-robust significance tests).

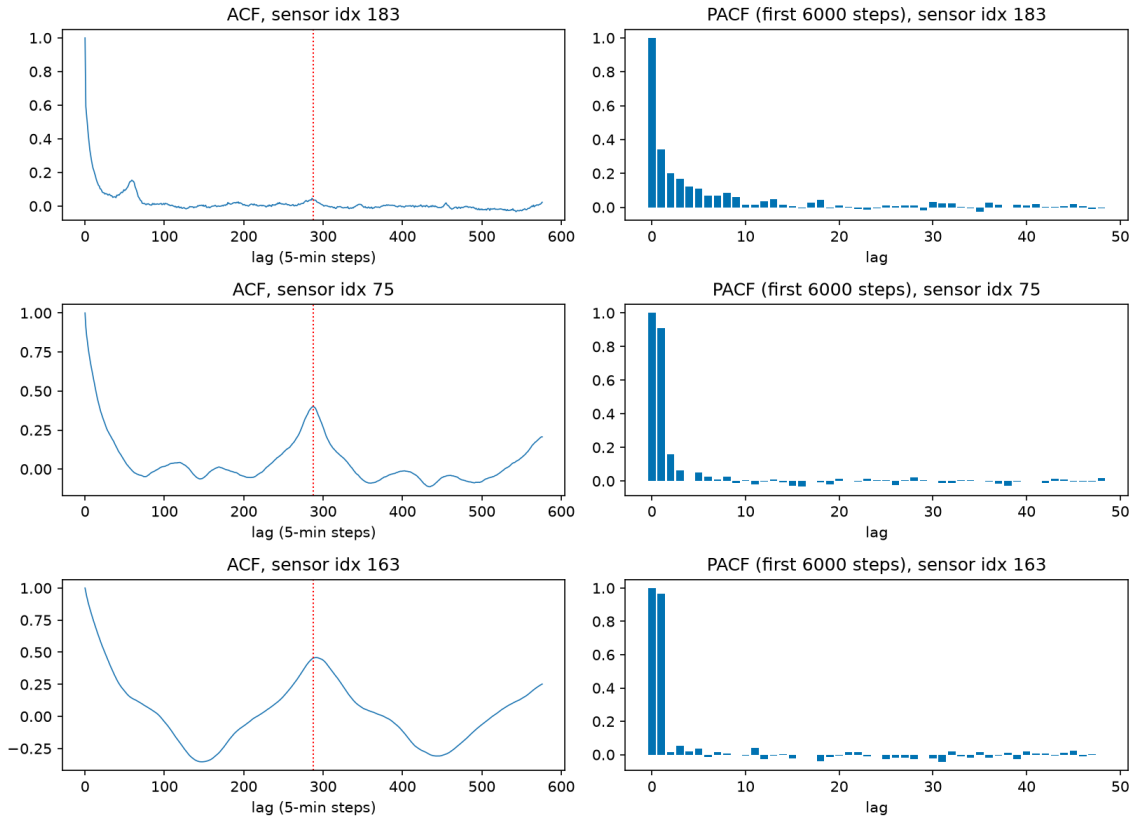


Figure 5. ACF (left, red line at lag 288) and PACF (right) for three sensors.

6.5 Spatial dependence

Hops	1	2	3	4	5	6	7	8	unreachable
Median correlation	0.446	0.245	0.194	0.179	0.164	0.150	0.140	0.144	0.196

Table 9. Median pairwise speed correlation by shortest-path distance in the (undirected) graph, training period.

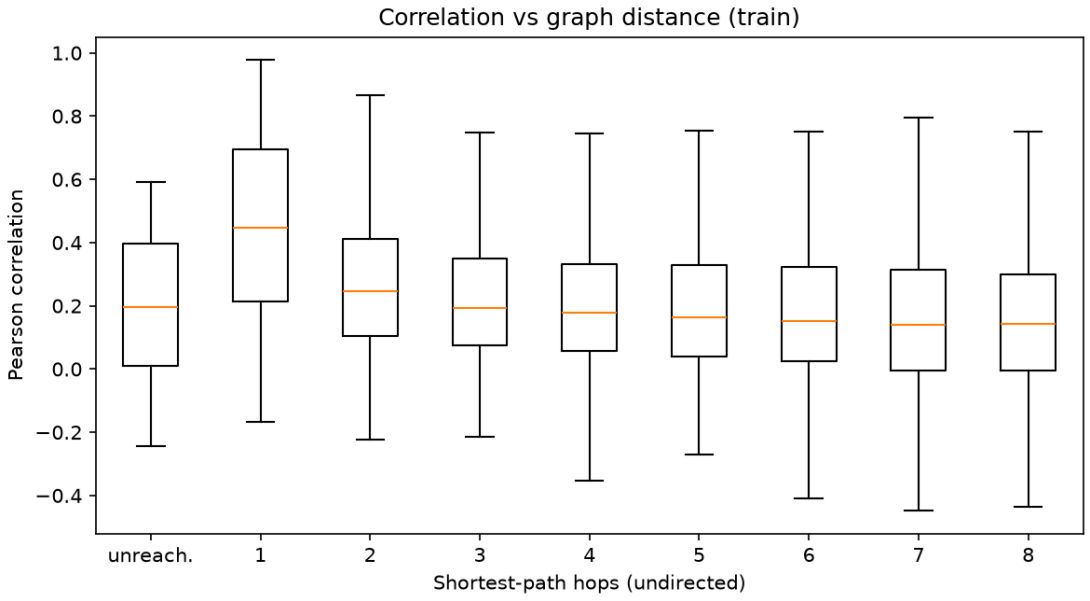


Figure 6. Correlation vs graph distance. Neighbours are clearly more informative than distant sensors.

6.6 Propagation evidence (lagged cross-correlation)

For every directed edge (i → j, j downstream) with both sensors < 10% missing, speeds are detrended by the training time-of-day mean and the correlation corr(z_down(t), z_up(t + lag)) is computed for lags −12…+12. A positive peak lag means the upstream sensor changes after the downstream one, i.e. congestion moving upstream.

Period	Pairs	Peak lag > 0	Peak lag = 0	Peak lag < 0
All times	1,228	38%	32%	30%
Weekday peak hours	1,228	50%	12%	38%

Table 10. Distribution of the peak lag over all adjacent pairs.

Reading: spatial dependence is clear; directional propagation is only modestly supported. The strongest pairs peak at lag 0 (adjacent sensors move together at 5-minute resolution); at peak hours positive lags outnumber negative ones, but many pairs show the opposite sign. Example pairs in Figure 9 are chosen by stated rules (top correlation; largest non-zero lag with correlation ≥ 0.5), never by eye.

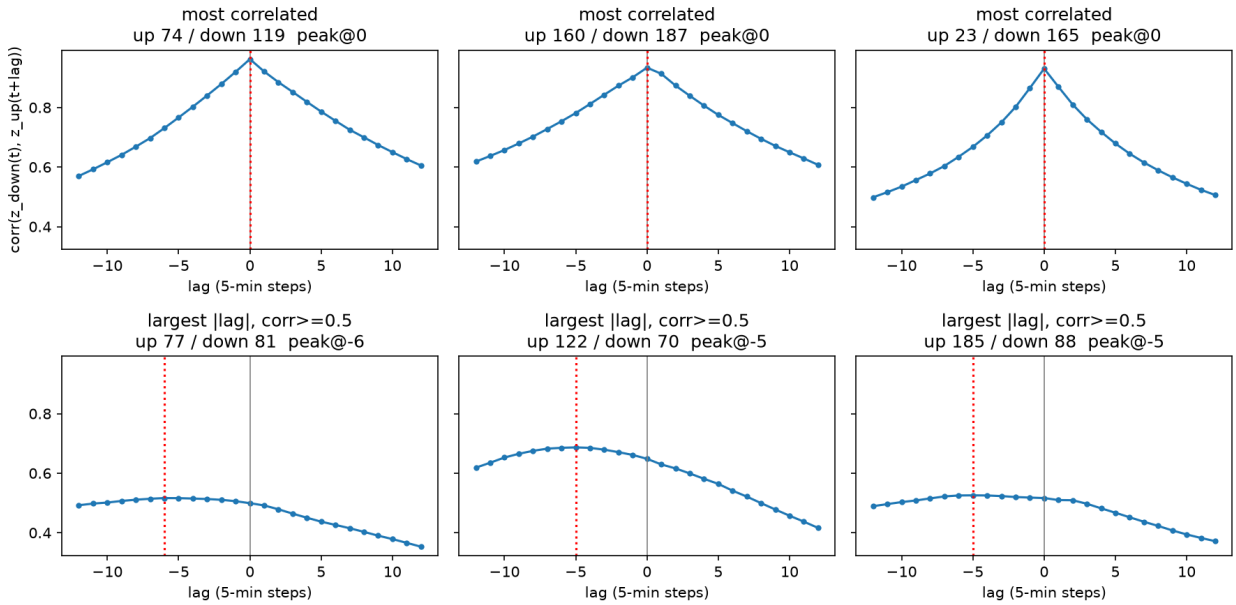


Figure 7. Cross-correlation curves for six pairs chosen by rule.

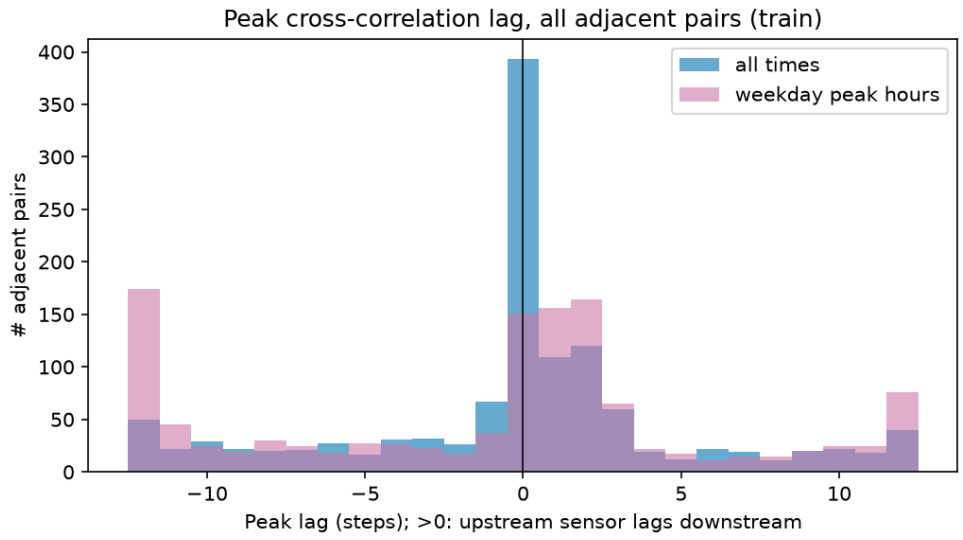


Figure 8. Peak lag across all adjacent pairs: all times vs weekday peak hours.

6.7 Where congestion happens

The median sensor is below 40 mph for 27% of weekday-peak time; 119 sensors exceed 20% and the worst is congested 100% of the time. 47 sensors have a mean speed below 55 mph. Interactive maps: `results/maps/eda_sensors.html`, `results/maps/eda_congestion_frequency.html`.

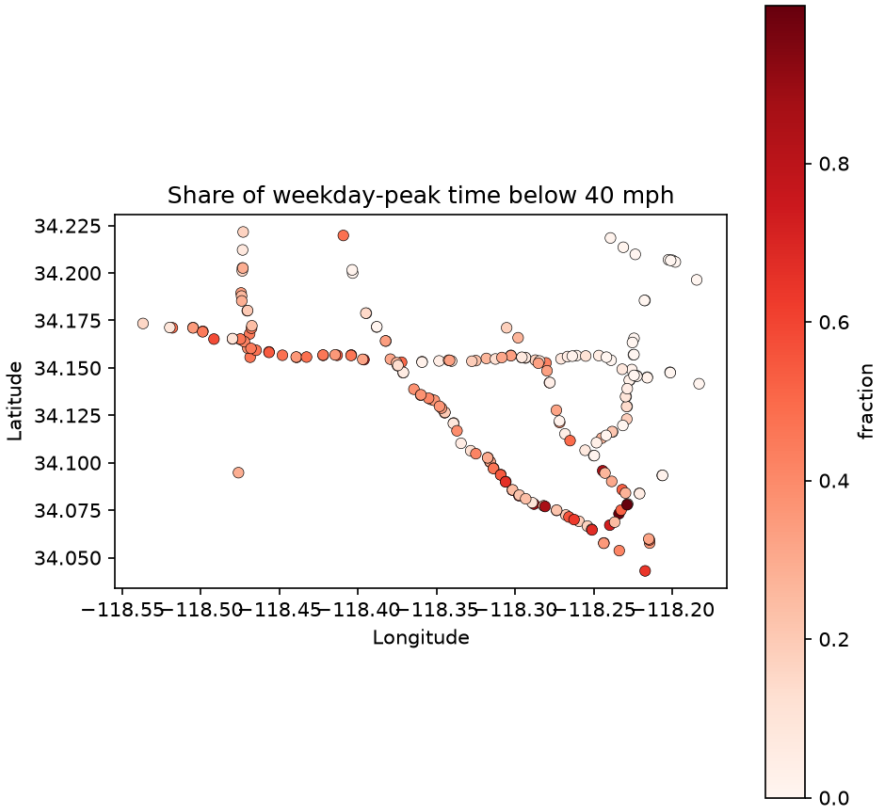


Figure 9. Share of weekday-peak time below 40 mph per sensor.

6.8 Split comparability

Split	Weekday share	Peak share	Missing rate	Mean speed (mph)
train	71.2%	23.7%	7.1%	58.58
val	66.4%	22.0%	7.0%	58.90
test	74.8%	25.2%	12.1%	57.77

Table 11. Composition of the three periods.

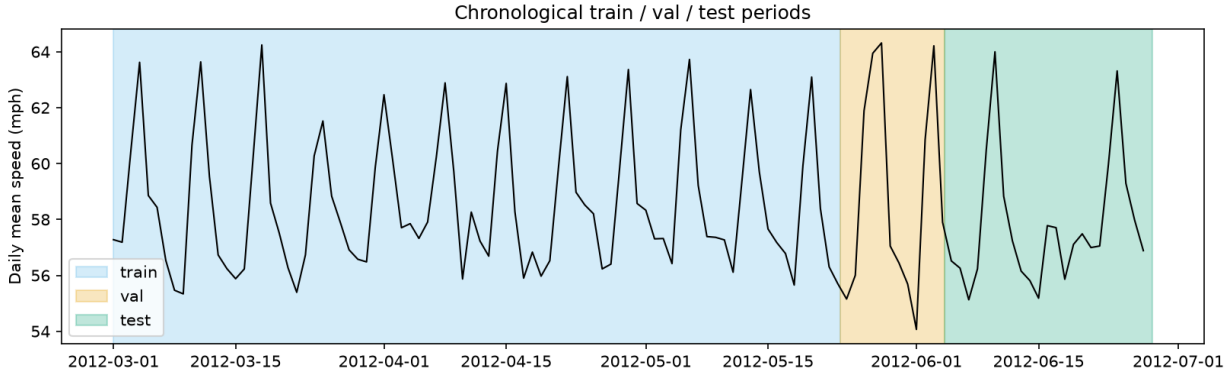


Figure 10. Daily mean speed with the train / validation / test periods.

7. Evaluation protocol (identical for all models)

7.1 Common prediction format

Every model writes `results/predictions/<model>[_seed<k>].npz` with `y_pred` and `y_true` [S, F, N] in mph, `mask` [S, F, N], `target_ts` [S, F], `sensor_ids` and metadata. The evaluator first asserts that every file has exactly the same test timestamps, masks and sensors as persistence (S = 6,832 test windows × 12 horizons × 207 sensors). Horizons a model does not produce (per-segment XGBoost trains only 15/30/60 minutes) are stored as NaN and skipped.

7.2 Metrics (masked, in mph)

$$\text{MAE} = \Sigma\, m \cdot |y - \hat{y}| \;/\; \Sigma\, m \qquad \text{RMSE} = \sqrt{(\,\Sigma\, m \cdot (y - \hat{y})^2 \;/\; \Sigma\, m\,)}$$
$$\text{MAPE} = 100 \cdot \Sigma\, m' \cdot |y - \hat{y}| / y \;/\; \Sigma\, m',\ m' = m \wedge (y \geq 1 \text{ mph})$$

An all-masked slice returns NaN, never 0. NaN predictions are removed from the mask. Hand-computed unit tests cover these cases (ADR-005).

7.3 Slices

- Every horizon 1…12; headline tables at 3, 6, 12 steps (15, 30, 60 minutes).
- Peak AM (06:00–10:00), peak PM (15:00–19:00), peak (either), off-peak weekday, weekend — judged on the target timestamp.
- Congested (true speed < 40 mph) vs free-flow (≥ 40 mph).
- Per sensor (for maps).

7.4 Statistical significance

- **Diebold–Mariano (DM) test** on the per-timestamp loss differential d_t = mean over sensors $|e_1|$ − mean $|e_2|$ (sensors observed in both), per headline horizon. Long-run variance: Newey–West with a Bartlett kernel and L = max(h − 1, ⌈T^(1/3)⌉) lags; Harvey–Leybourne–Newbold small-sample correction; Student t with T − 1 degrees of freedom. Negative statistic: the first model has lower loss.
- For multi-seed models the test is run per seed pair; we report the median statistic and the **largest** (most conservative) p-value.
- **Day-block bootstrap:** whole days of target timestamps are resampled with replacement (1,000 draws, seed 42) for 95% intervals; paired versions resample the same days for both models so day difficulty cancels.

7.5 Automatic sanity checks

- Any model with MAE at 60 minutes below 3.0 mph triggers a leakage warning (published strong graph models report ≈ 3.5).
- Any learned model not beating persistence at 15 minutes triggers “something is broken”.
- Final run: “no warnings”.

8. Baselines

- **Persistence:** $\hat{y}(t + h)$ = last observed value in the input window (past-only forward fill up to 11 steps; time-of-day mean if the whole window is missing).
- **Historical average:** mean of that sensor at the same (day-of-week, 5-minute slot), training period only ($7 \times 288 \times 207$ table); falls back to the slot mean.
- **Seasonal naive:** $\hat{y}(t + h) = y(t + h - 288)$ (same time yesterday); when that value is missing it falls back to the historical average (12.61% of entries).

Baseline	MAE 15	MAE 30	MAE 60	RMSE 15	RMSE 30	RMSE 60	MAPE 15	MAPE 30	MAPE 60
Persistence	3.49	4.20	5.37	6.39	8.02	10.30	8.60	10.83	14.48
Historical average	4.19	4.19	4.19	7.86	7.86	7.86	13.06	13.06	13.06
Seasonal naive	5.50	5.50	5.50	10.71	10.71	10.71	15.57	15.57	15.57

Table 12. Baseline test errors (mph; MAPE in %).

The historical average is flat across horizons because it ignores the current state; its 4.19 mph is close to the 4.16 reported for the historical average in the DCRNN paper, a useful external check of the pipeline.

9. Phase 1 — machine learning

9.1 Tabular features

One row per (sensor, anchor time); all features use data at or before the anchor (tested by perturbing the future).

Group	Features	Count
Lags	lag_0 … lag_11 (imputed speed)	12
Rolling statistics	mean, std, min, max over 3, 6, 12 steps	12
Differences	lag_0 − lag_1, lag_0 − lag_3	2
Missing flags	miss_0 … miss_11 and n_missing_window	13
Calendar (at anchor)	tod_sin, tod_cos, dow_sin, dow_cos, is_weekend, is_holiday	6
Static (pooled models)	sensor_idx (categorical), latitude, longitude, in_degree, out_degree	5
Neighbour (XGB-spatial)	upstream and downstream weighted means at lags 0–2 (6), 2-hop mean, own − upstream, own − downstream	9
Daily lag (side experiment)	lag_288 (same time yesterday)	1

Table 13. Feature groups. Per-segment models use 45 features, pooled models 50, XGB-spatial 59, the daily-lag run 51.

Neighbour features use row-normalised $\mathtt{W}$ (downstream senders) and $\mathtt{W}^\top$ (upstream senders); sensors with no neighbour in a direction get NaN, which XGBoost handles natively.

9.2 Models

- **XGBoost per-segment:** one model per sensor per headline horizon, 207 × 3 = 621 models, ~24k rows each, `tree_method=hist`, single-threaded per model and parallel over sensors.
- **XGBoost global (pooled):** one model per horizon (12 models) across all sensors with sensor id as a categorical feature.
- **Random Forest (multi-output):** one pooled forest that predicts all 12 horizons jointly (joint squared-error split criterion); rows with any missing horizon are dropped.
- **XGBoost spatial:** the pooled model plus the 9 neighbour features — the bridge experiment that tests how much of the graph benefit hand-made features capture.

9.3 Hyperparameter tuning (Optuna, validation only)

TPE sampler (seed 42). XGBoost objective: validation MAE at horizon 6 on 300,000 sampled training rows and 150,000 validation rows, early stopping 30 rounds, tree cap 400; per-segment tuning pools 20 random sensors with shared hyperparameters. Random Forest: mean validation MAE over all 12 horizons on 60,000 / 30,000 rows. The test split is never touched during tuning.

Model	Trials	Best val MAE	Median val MAE
XGBoost global (pooled)	15	3.1530	3.2035
XGBoost spatial	15	3.0874	3.1451
XGBoost per-segment	15	3.5945	3.6220
Random Forest (multi-output)	6	3.5295	3.5643

Table 14. Tuning summary (validation MAE in mph; tuning subsets differ between models, so values are not mutually comparable).

Parameter	Search range	Per-segment	Global	Spatial
max_depth	3 – 10	9	10	10
learning_rate	0.02 – 0.3 (log)	0.0343	0.0402	0.0402
subsample	0.5 – 1.0	0.5028	0.5435	0.5435
colsample_bytree	0.5 – 1.0	0.9077	0.9238	0.9238
min_child_weight	1 – 20	15	6	6
reg_lambda	0.001 – 10 (log)	0.8242	5.2222	5.2222
gamma	0 – 5	3.8564	3.7865	3.7865
objective	squared error | absolute error | pseudo-Huber	reg:absoluteerror	reg:absoluteerror	reg:absoluteerror

Table 15. XGBoost search space and the selected values (absolute-error loss chosen for all three). Global and spatial chose identical values: same seed and the same best trial index.

Random Forest search: n_estimators 60–150, max_depth 8–30, max_features ∈ {0.2, 0.33, 0.5}, min_samples_leaf 1–20, max_samples 0.5. Selected: 135 trees, max_depth 12, max_features 0.5, min_samples_leaf 11, max_samples 0.5.

9.4 Training budget (reduced for a laptop, ADR-006)

Model	Training rows	Training time	Notes
XGBoost global	≈ 1.15 million per horizon (25% sample)	755 s	12 horizons; every model used 398–399 of 400 trees
XGBoost spatial	≈ 1.15 million per horizon (25% sample)	1252 s	12 horizons; every model used 398–399 of 400 trees
XGBoost per-segment	all rows of each sensor	198 s	621 models
Random Forest	444,904 of 4,449,042 complete rows (10%)	56 s	10.3% of rows dropped (a horizon missing)
XGBoost global + daily lag	≈ 1.15 million per horizon	1173 s	side experiment

Table 16. Training data and wall-clock time (6 CPU workers, low priority, Apple M3 Pro).

Early stopping never triggered for the pooled XGBoost models: the best iteration was 398–399 of the 400-tree cap at every horizon. More trees (and more training rows) would very likely lower their error further. The reported ML numbers are therefore conservative.

9.5 Phase 1 results

Model	MAE 15	MAE 30	MAE 60	RMSE 15	RMSE 30	RMSE 60	MAPE 15	MAPE 30	MAPE 60
Persistence	3.49	4.20	5.37	6.39	8.02	10.30	8.60	10.83	14.48
Historical average	4.19	4.19	4.19	7.86	7.86	7.86	13.06	13.06	13.06
Seasonal naive	5.50	5.50	5.50	10.71	10.71	10.71	15.57	15.57	15.57
XGBoost per-segment	2.86	3.28	3.68	5.60	6.62	7.52	7.92	9.77	11.65
XGBoost global (pooled)	2.88	3.33	3.78	5.69	6.80	7.76	7.87	9.90	12.11
Random Forest (multi-output)	3.14	3.80	4.73	5.70	6.89	8.25	8.50	10.87	14.27
XGBoost spatial	2.82	3.26	3.73	5.46	6.54	7.54	7.63	9.53	11.82

Table 17. Phase 1 test errors (mph; MAPE in %).

- XGB-spatial is best at 15 minutes (2.820) and 30 minutes (3.258); per-segment XGBoost is best at 60 minutes (3.684).
- The forest is the weakest ML model (3.137 / 3.798 / 4.732); at 60 minutes it loses to the historical average.
- All Phase 1 ML models beat persistence at every horizon; none comes close to the 3.0 mph leakage bound at 60 minutes.

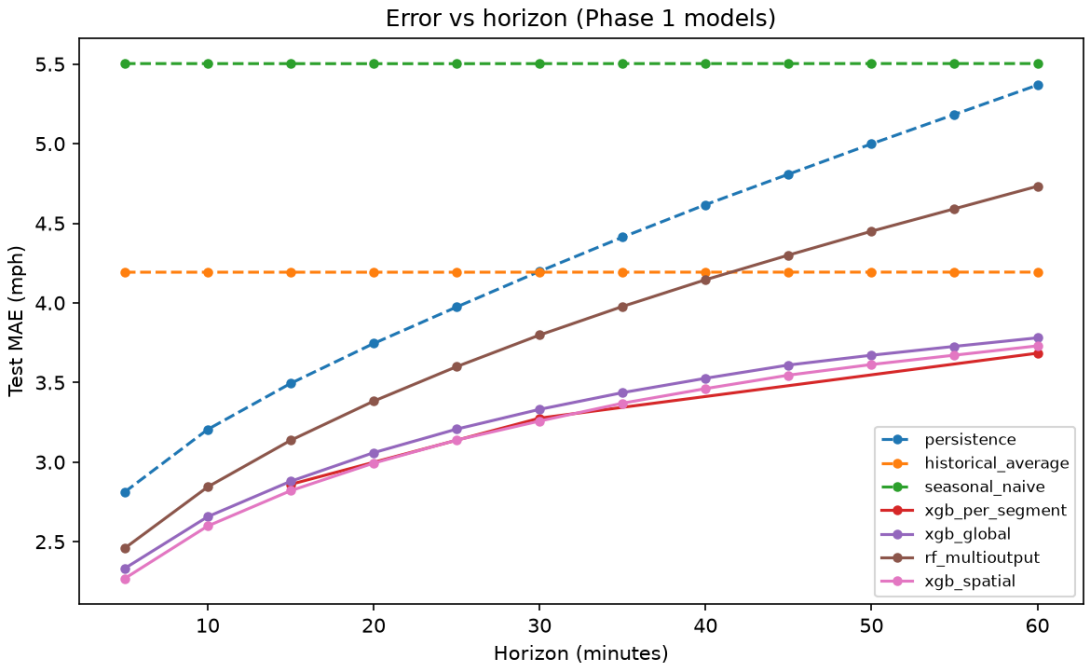


Figure 11. Test MAE vs horizon for baselines and Phase 1 models.

9.6 What drives the trees (gain and TreeSHAP)

TreeSHAP values come from XGBoost's exact `pred_contribs` on 5,000 seeded test rows with observed targets.

Model	Horizon	Top 8 features by mean |SHAP| (mph)
XGBoost global (pooled)	15 min	roll_mean_6 (3.79), lag_0 (2.45), roll_max_3 (1.47), roll_max_6 (1.35), roll_mean_3 (0.89), sensor_idx (0.72), tod_cos (0.45), tod_sin (0.24)
XGBoost global (pooled)	60 min	roll_mean_6 (2.43), sensor_idx (1.77), lag_0 (1.45), tod_cos (1.44), roll_mean_3 (1.19), roll_max_3 (1.18), tod_sin (0.81), dow_sin (0.46)
XGBoost spatial	15 min	roll_mean_6 (3.32), lag_0 (2.40), roll_max_3 (1.36), roll_mean_3 (1.16), roll_max_6 (1.16), sensor_idx (0.82), up_mean_lag0 (0.46), down_mean_lag0 (0.44)
XGBoost spatial	60 min	roll_mean_6 (2.32), sensor_idx (1.79), lag_0 (1.50), roll_mean_3 (1.28), tod_cos (1.27), roll_max_3 (0.89), tod_sin (0.77), up_mean_lag0 (0.39)

Table 18. Most important features.

Short-window averages and the latest speed dominate at 15 minutes; sensor identity and time of day gain weight at 60 minutes. Neighbour speeds enter the top features of XGB-spatial but never dominate.

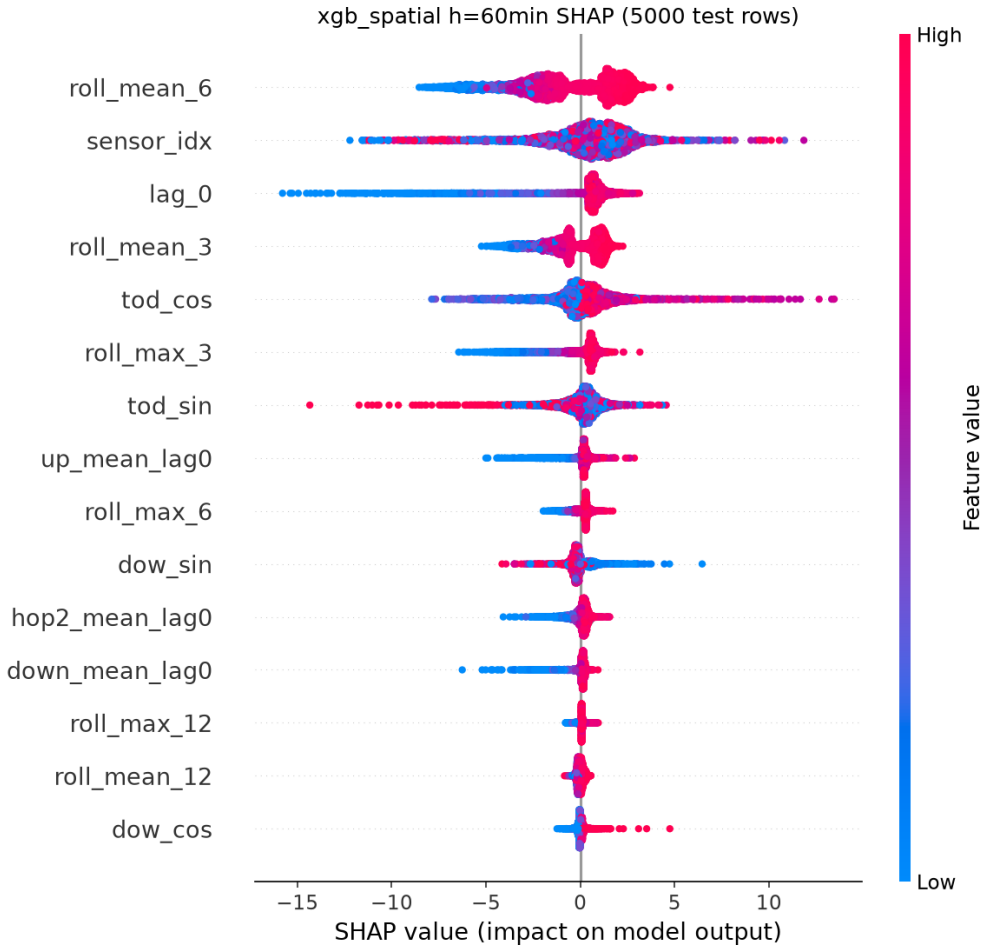


Figure 12. TreeSHAP summary, XGB-spatial at 60 minutes.

9.7 Gap analysis: what do neighbour features add?

Horizon	Slice	MAE global	MAE spatial	Δ MAE	95% CI	DM p
15 min	all	2.880	2.820	+0.059	[+0.051, +0.069]	0.000
15 min	peak	3.866	3.749	+0.117	[+0.101, +0.135]	0.000
15 min	offpeak_weekday	2.579	2.540	+0.038	[+0.031, +0.048]	0.000
15 min	weekend	2.356	2.320	+0.036	[+0.025, +0.057]	0.000
15 min	congested	7.513	7.177	+0.336	[+0.282, +0.392]	0.006
15 min	freeflow	2.226	2.205	+0.020	[+0.017, +0.024]	0.000
30 min	all	3.331	3.258	+0.073	[+0.059, +0.090]	0.000
30 min	peak	4.730	4.578	+0.152	[+0.126, +0.182]	0.000
30 min	offpeak_weekday	2.863	2.816	+0.048	[+0.034, +0.065]	0.000
30 min	weekend	2.697	2.661	+0.037	[+0.024, +0.057]	0.000
30 min	congested	10.147	9.588	+0.559	[+0.452, +0.670]	0.000
30 min	freeflow	2.369	2.364	+0.005	[-0.003, +0.012]	0.764
60 min	all	3.781	3.730	+0.051	[+0.033, +0.072]	0.000
60 min	peak	5.654	5.542	+0.112	[+0.081, +0.143]	0.000
60 min	offpeak_weekday	3.113	3.081	+0.032	[+0.012, +0.055]	0.000
60 min	weekend	3.050	3.030	+0.020	[+0.000, +0.041]	0.062
60 min	congested	13.034	12.576	+0.458	[+0.343, +0.570]	0.001
60 min	freeflow	2.475	2.482	-0.007	[-0.018, +0.004]	0.038

Table 19. Δ = MAE(global) − MAE(spatial); positive means the spatial features help. Paired day-block bootstrap intervals; Diebold–Mariano p-values.

The spatial features help in almost every slice, most at peaks and in congested states (≈ 0.3–0.6 mph in congestion), as the propagation hypothesis predicts — but the gain does **not** grow with horizon (largest at 30 minutes), so the "graph helps more at long horizons" part is only partly supported. In free flow at 60 minutes they are neutral to slightly harmful.

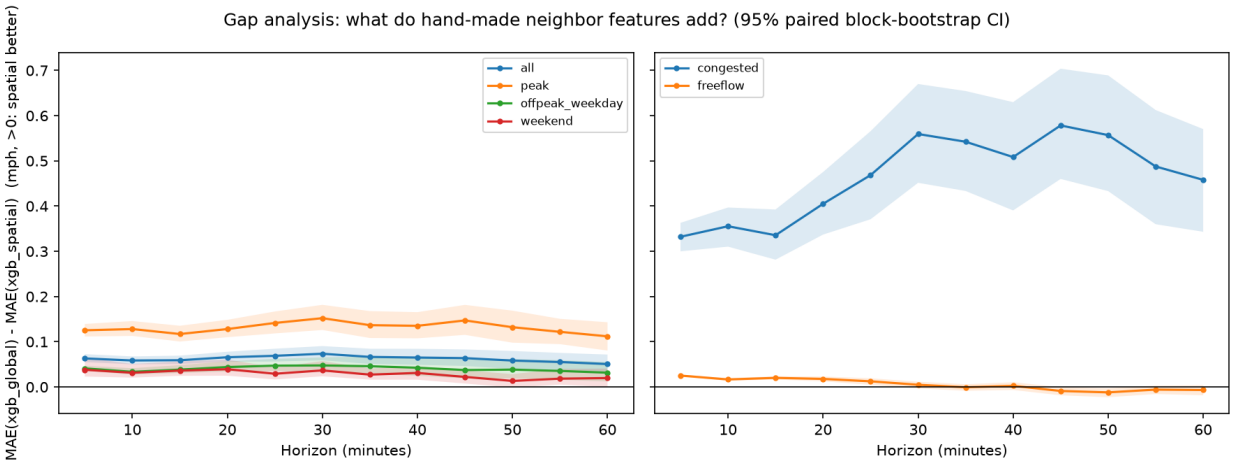


Figure 13. Gap analysis by horizon and slice with 95% intervals.

9.8 Side experiment: same-time-yesterday lag

Model	MAE 15	MAE 30	MAE 60
XGBoost global (pooled)	2.8795	3.3313	3.7805
XGBoost global + daily lag	2.8788	3.3295	3.7849

Table 20. Adding lag_288 changes nothing material (kept out of the main equal-window comparison).

9.9 Hardest sensors

Sensor	Latitude	Longitude	MAE @ 60 min (XGB global)
717462	34.08993	-118.30607	9.30
717468	34.09699	-118.31381	9.03
717466	34.09359	-118.30918	9.01
773939	34.15297	-118.37226	8.67
717472	34.10045	-118.31601	8.61

Table 21. The five hardest sensors cluster at one location (≈ 34.09 N, 118.31 W).

10. Phase 2 — deep learning

10.1 The graph attention layer

For node i with neighbourhood N(i) ∪ {i} (self-loops always included), with K heads of size D:

$$h' = W\ h\ e_ij = \mathrm{LeakyReLU}(\ a_dst^{\top}\ h'_i + a_src^{\top}\ h'_j\ )\ (= a^{\top}[h'_i \parallel h'_j])\ \alpha_ij = \mathrm{softmax_j}(e_ij)\ out_i = \Sigma_j\ \alpha_ij\ h'_j + b$$

- **Dense masked form** (`DenseGATLayer`): scores for all [N × N] pairs, non-neighbours set to $-\infty$ before the softmax; heads concatenated (or averaged).
- **Sparse form** (`SparseGATLayer`): padded neighbour lists (D_max = 26 for the bidirectional graph vs N = 207); numerically identical to the dense layer (tested).
- **Variants:** GATv2 (dynamic attention, $a^{\top}$ LeakyReLU(W_dst h_i + W_src h_j), computed in gradient-checkpointed chunks of 32 graphs), fixed row-normalised weights ("mean", GCN-style), and an edge-weight bias $\beta \cdot$W_ij per head.
- **Verification:** outputs and attention weights equal PyTorch Geometric's `GATConv` (concat and mean) and `GATv2Conv` within 1e-5 on a random 12-node graph with copied weights (`tests/test_gat_layer.py`). PyG's edge_index is (source j, target i), so it is built as `adj.nonzero().flip(1).T`.

10.2 STGAT architecture

- Input X [B, 12, 207, 8] → Linear 8 → d = 64 → plus a learnable node embedding E ∈ ℝ^(207×64).
- 2 spatio-temporal blocks. Each: gated causal temporal convolution (kernel 3, dilation 1 then 2, GLU gate, left padding so step t sees only ≤ t) → multi-head GAT (4 heads × 16 = 64) on the **last 4 of 12 time steps** → ELU → dropout 0.1 → residual → LayerNorm.
- Attention dropout 0.1; graph `bidirectional` (W + $W^{\top}$); readout = last time step; decoder Linear 64 → 64 → ReLU → dropout → Linear 64 → 12.
- Non-autoregressive: all 12 horizons predicted at once.
- Parameters: **77,004**.

10.3 GRU no-graph ablation

- Same embedding, node embedding, width d = 64, dropout and decoder; a 2-layer GRU runs over each sensor's own 12 steps; no spatial layer.
- Parameters: **68,684** (similar capacity), so any gap to STGAT is attributable to the graph, not to deep learning in general.

10.4 Training

Setting	Value
Loss	masked MAE in scaled space (normalised by the number of observed targets)
Optimiser	Adam, learning rate 0.001, weight decay 0.0001
Schedule	ReduceLROnPlateau on validation MAE (factor 0.5, patience 5)
Gradient clipping	norm 5.0
Batch size	64
Epochs	max 40, early stopping patience 8 on validation MAE in mph; best weights restored
Epoch fraction	0.34 (each epoch sees a random third of the training windows)
Seeds	0, 1, 2
Device	Apple M3 Pro GPU (PyTorch MPS), 4 CPU threads, low priority

Table 22. Deep-learning training configuration (same for STGAT and GRU).

The deep models were not hyperparameter-tuned. Width 64, 4 heads, 2 blocks, dropout 0.1, learning rate 1e-3 and weight decay 1e-4 were set by hand; they were probed only by the one-factor ablations in Section 13. Only the tree models were tuned (Optuna, Section 9.3).

10.5 Compute study and simplification (ADR-007)

Configuration	Time per iteration	Time per epoch
STGAT dense, GAT on all 12 steps, CPU	4.34 s	27.1 min
STGAT dense, GAT on all 12 steps, MPS	0.59 s	3.7 min
STGAT sparse neighbour lists, all 12 steps, MPS	0.66 s	4.1 min (no gain: gathers dominate)
STGAT GAT on last 4 steps + 34% of windows per epoch, MPS	0.28 s	≈ 0.6 min (+ validation)
GRU no-graph, CPU / MPS (full epoch)	0.27 s / 0.08 s	1.7 / 0.5 min

Table 23. Measured training speed that motivated the simplification.

Profiling showed the per-block cost was dominated by the graph attention (0.26 s of 0.33 s per block), which is memory-bound. Applying attention on the most recent 4 steps (20 minutes of neighbour state) and sampling a third of the highly redundant windows each epoch cut the cost about six-fold, to ≈ 28 minutes per seed.

10.6 Training outcomes

Model	Seed	Epochs run	Best epoch	Best val MAE (mph)	Train time
GRU no-graph	0	40	34	3.0136	490 s
GRU no-graph	1	40	40	3.0088	496 s
GRU no-graph	2	33	25	3.0162	413 s
STGAT	0	40	40	2.9817	1692 s
STGAT	1	40	40	2.9947	1685 s
STGAT	2	40	35	2.9802	1687 s
Hybrid: residual	0	40	40	2.9312	1391 s
Hybrid: residual	1	40	32	2.9332	1379 s
Hybrid: residual	2	40	40	2.9335	1383 s

Table 24. Per-seed training results. STGAT never triggered early stopping within 40 epochs (it was still improving).

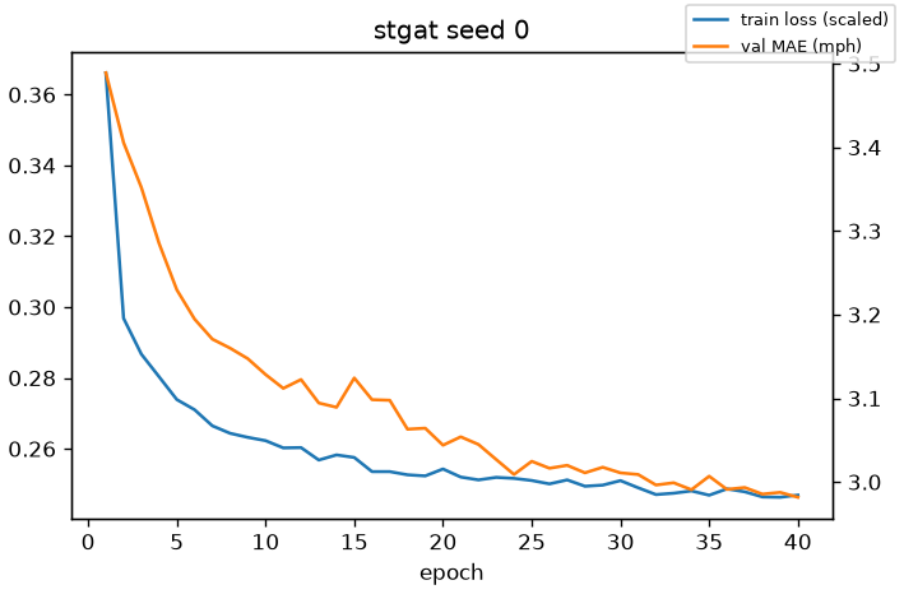


Figure 14. Training curve, STGAT seed 0 (training loss in scaled units; validation MAE in mph).

10.7 Results

Model	MAE 15	MAE 30	MAE 60	RMSE 15	RMSE 30	RMSE 60
XGBoost per-segment	2.860	3.276	3.684	5.601	6.616	7.516
XGBoost global (pooled)	2.880	3.331	3.781	5.691	6.801	7.762
XGBoost spatial	2.820	3.258	3.730	5.465	6.544	7.544
GRU no-graph	2.892 ± 0.007	3.313 ± 0.013	3.761 ± 0.036	5.618 ± 0.032	6.655 ± 0.036	7.710 ± 0.095
STGAT	2.901 ± 0.011	3.276 ± 0.021	3.666 ± 0.022	5.524 ± 0.010	6.483 ± 0.007	7.436 ± 0.072

Table 25. Deep models vs the tree models (mph; ± std over 3 seeds).

- STGAT beats persistence at every horizon (acceptance criterion met).
- Graph vs no graph: equal at 15 minutes (2.901 vs 2.892, DM not significant), slightly ahead at 30 (3.276 vs 3.313, not significant), clearly ahead at 60 (3.666 vs 3.761, significant).
- Deep vs trees: STGAT is slightly worse than pooled XGBoost at 15 minutes (2.901 vs 2.880) and better at 60 (3.666 vs 3.781); per-segment XGBoost stays very close at 60 minutes (3.684).
- STGAT has a lower RMSE than every tree model at 30 and 60 minutes, i.e. fewer large misses.

11. Hybrid models

11.1 Why a residual hybrid

XGBoost captures local autoregression and calendar effects cheaply; spatial propagation is its blind spot. The residual hybrid trains STGAT to learn only what XGBoost misses:

$$\hat{y}_\text{hybrid}(t + h, i) = \hat{y}_\text{XGB}(t + h, i) + g_\theta(X_\text{window}, \hat{y}_\text{XGB}, G)(h, i)$$

STGAT receives the scaled XGBoost forecast (12 values per sensor) concatenated to its readout before the decoder, and the last decoder layer is zero-initialised, so training starts exactly at the XGBoost forecast (tested).

11.2 Leakage-safe out-of-fold base forecasts

- XGBoost's predictions on its own training rows are overfit; their residuals are unrealistically small. The residual model must learn from **out-of-fold (OOF)** forecasts instead.
- Forward-chaining over the 23,967 training windows in 5 chronological blocks with edges [0, 4793, 9586, 14380, 19173, 23967].
- For blocks 2–5, an XGBoost model (tuned global parameters, 15% row sample, early stopping on 20,000 validation rows) trained on the earlier blocks predicts the block; a gap of $H + F - 1 = 23$ anchors separates training targets from the block's inputs.
- Block 1 gets no OOF forecast and is dropped: residual training uses 19,174 windows (80.0% coverage).
- The final XGBoost models (trained on all training rows) provide the base forecasts for validation and test.
- Caveat: early-block models see less data, so OOF errors are slightly larger than the final model's — the conservative direction.

11.3 Gating ensemble and simple average

- **Gate:** $\hat{y} = w \cdot \hat{y}_\text{STGAT} + (1 - w) \cdot \hat{y}_\text{XGB}$ with $w = \text{sigmoid}(\text{MLP}(16 \to 16 \to 1))$ of horizon one-hot (12), weekday-peak flag, time-of-day sin/cos of the target, and the input window's missing rate; trained with Adam (lr 0.01) on the first 80% of validation windows, early-stopped (patience 30, ≤ 400 steps) on the last 20% (ADR-008: validation is then not fully independent).
- **Average:** $0.5 \cdot \hat{y}_\text{XGB} + 0.5 \cdot \hat{y}_\text{STGAT}$, per seed.
- All hybrids: 3 seeds, paired with the STGAT seed.

11.4 Results

Model	MAE 15	MAE 30	MAE 60	RMSE 15	RMSE 30	RMSE 60
XGBoost global (pooled)	2.880	3.331	3.781	5.691	6.801	7.762
STGAT	2.901 ± 0.011	3.276 ± 0.021	3.666 ± 0.022	5.524 ± 0.010	6.483 ± 0.007	7.436 ± 0.072
Hybrid: residual	2.820 ± 0.005	3.219 ± 0.002	3.647 ± 0.009	5.461 ± 0.029	6.453 ± 0.033	7.456 ± 0.021
Hybrid: gating	2.836 ± 0.005	3.233 ± 0.012	3.607 ± 0.018	5.522 ± 0.008	6.448 ± 0.012	7.344 ± 0.063
Hybrid: average	2.840 ± 0.005	3.246 ± 0.010	3.640 ± 0.018	5.525 ± 0.007	6.506 ± 0.013	7.392 ± 0.058

Table 26. Hybrids vs their components (mph; ± std over 3 seeds).

Component	Slice	Gain 15	Gain 30	Gain 60
STGAT	all	+0.081	+0.058	+0.019
STGAT	peak	+0.106	+0.084	-0.091
STGAT	offpeak_weekday	+0.073	+0.047	+0.056
STGAT	weekend	+0.069	+0.049	+0.064
STGAT	congested	+0.258	+0.327	-0.591
STGAT	freeflow	+0.056	+0.020	+0.105
XGBoost global (pooled)	all	+0.059	+0.113	+0.133
XGBoost global (pooled)	peak	+0.146	+0.304	+0.374
XGBoost global (pooled)	offpeak_weekday	+0.027	+0.041	+0.037

Component	Slice	Gain 15	Gain 30	Gain 60
XGBoost global (pooled)	weekend	+0.027	+0.046	+0.066
XGBoost global (pooled)	congested	+0.609	+1.283	+1.536
XGBoost global (pooled)	freeflow	-0.018	-0.052	-0.065

Table 27. Gain of the residual hybrid = MAE(component) − MAE(hybrid), seed-averaged (positive: hybrid better).

Over XGBoost the residual hybrid's gain grows with horizon and is largest in congestion and at peaks; in free flow at 60 minutes it is slightly negative. Over STGAT the gain shrinks with horizon: STGAT already captures much of the local structure at long range.

Horizon (min)	5	10	15	20	25	30	35	40	45	50	55	60
Weight on STGAT, peak	0.35	0.49	0.56	0.65	0.72	0.79	0.82	0.85	0.85	0.84	0.82	0.79
Weight on STGAT, off-peak	0.18	0.27	0.33	0.36	0.40	0.45	0.49	0.50	0.52	0.49	0.45	0.42

Table 28. Mean learned gate weight on STGAT (test set, averaged over seeds).

The gate trusts STGAT more at peaks than off-peak and more at longer horizons — exactly the pattern the hypothesis predicts.

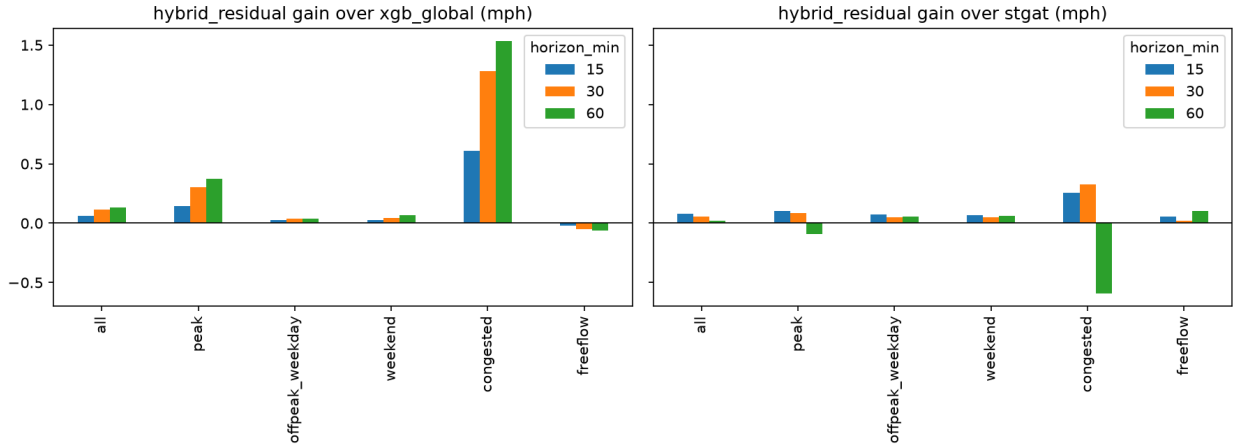


Figure 15. Residual-hybrid gain over XGBoost-global and over STGAT by slice and horizon.

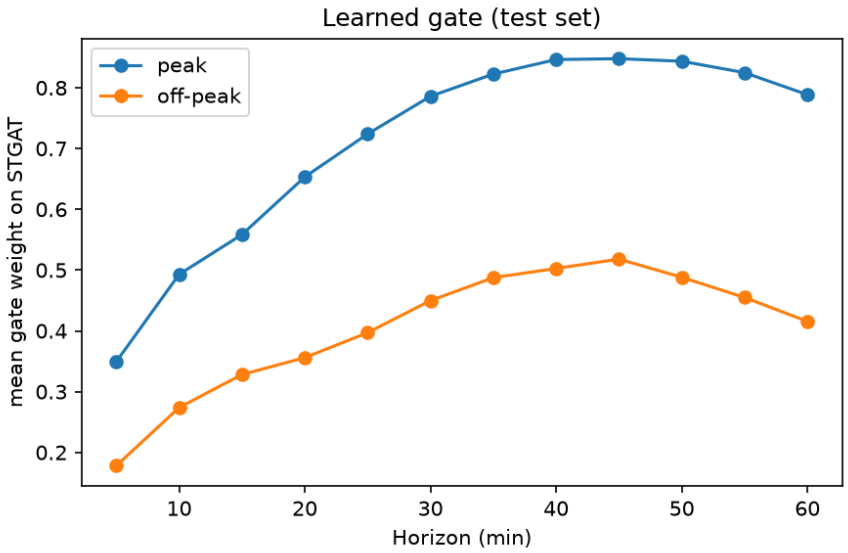


Figure 16. Learned gate weight on STGAT by horizon, peak vs off-peak.

12. All results

12.1 Main table

Model	MAE 15	MAE 30	MAE 60	RMSE 15	RMSE 30	RMSE 60	MAPE 15	MAPE 30	MAPE 60
Persistence	3.494	4.198	5.368	6.394	8.016	10.296	8.597	10.826	14.479
Historical average	4.193	4.193	4.193	7.862	7.862	7.862	13.059	13.059	13.059
Seasonal naive	5.503	5.503	5.503	10.711	10.711	10.711	15.573	15.573	15.573
XGBoost per-segment	2.860	3.276	3.684	5.601	6.616	7.516	7.916	9.773	11.652
XGBoost global (pooled)	2.880	3.331	3.781	5.691	6.801	7.762	7.871	9.899	12.112
Random Forest (multi-output)	3.137	3.798	4.732	5.704	6.892	8.251	8.501	10.875	14.268
XGBoost spatial	2.820	3.258	3.730	5.465	6.544	7.544	7.633	9.526	11.818
GRU no-graph	2.892	3.313	3.761	5.618	6.655	7.710	7.835	9.670	11.533
STGAT	2.901	3.276	3.666	5.524	6.483	7.436	7.765	9.407	10.970
Hybrid: residual	2.820	3.219	3.647	5.461	6.453	7.456	7.507	9.184	11.217
Hybrid: gating	2.836	3.233	3.607	5.522	6.448	7.344	7.677	9.383	11.029
Hybrid: average	2.840	3.246	3.640	5.525	6.506	7.392	7.694	9.512	11.323

Table 29. All models, all headline metrics (seed means for deep models).

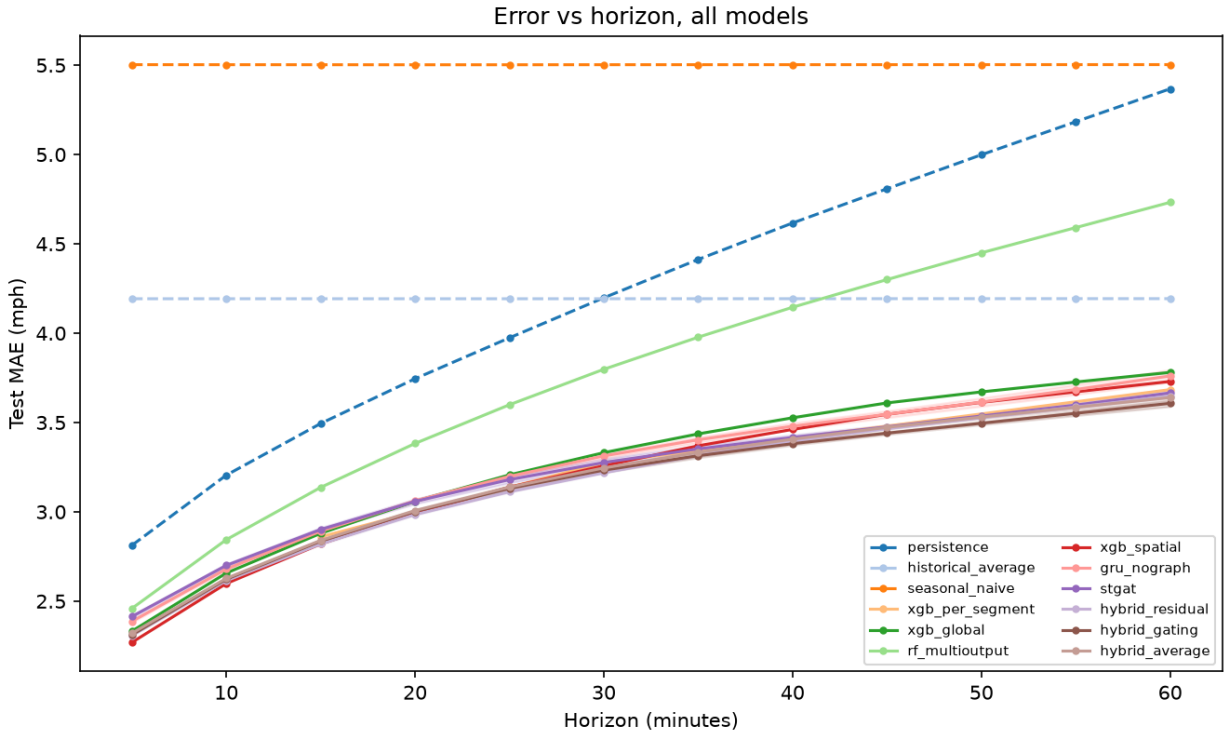


Figure 17. MAE vs horizon for every model (bands: ± std over seeds).

12.2 Every horizon

Model	5	10	15	20	25	30	35	40	45	50	55	60
Persistence	2.81	3.21	3.49	3.74	3.97	4.20	4.41	4.62	4.81	5.00	5.18	5.37
Historical average	4.19	4.19	4.19	4.19	4.19	4.19	4.19	4.19	4.19	4.19	4.19	4.19
Seasonal naive	5.50	5.50	5.50	5.50	5.50	5.50	5.50	5.50	5.50	5.50	5.50	5.50
XGBoost per-segment	–	–	2.86	–	–	3.28	–	–	–	–	–	3.68
XGBoost global (pooled)	2.33	2.66	2.88	3.06	3.21	3.33	3.44	3.53	3.61	3.67	3.73	3.78
Random Forest (multi-output)	2.46	2.84	3.14	3.38	3.60	3.80	3.98	4.15	4.30	4.45	4.59	4.73
XGBoost spatial	2.27	2.60	2.82	2.99	3.14	3.26	3.37	3.46	3.55	3.61	3.67	3.73
GRU no-graph	2.39	2.68	2.89	3.06	3.20	3.31	3.40	3.48	3.55	3.61	3.68	3.76
STGAT	2.41	2.70	2.90	3.06	3.18	3.28	3.35	3.42	3.48	3.54	3.60	3.67
Hybrid: residual	2.31	2.61	2.82	2.98	3.11	3.22	3.32	3.40	3.47	3.53	3.59	3.65
Hybrid: gating	2.31	2.62	2.84	3.00	3.13	3.23	3.31	3.38	3.44	3.50	3.55	3.61
Hybrid: average	2.32	2.63	2.84	3.01	3.14	3.25	3.33	3.41	3.47	3.53	3.58	3.64

Table 30. MAE (mph) at every horizon in minutes.

12.3 Peak, off-peak and weekend

Model	Peak 15	Peak 60	Off-peak 15	Off-peak 60	Weekend 15	Weekend 60
Persistence	4.39	7.45	3.29	4.85	2.86	3.98
Historical average	6.04	6.06	3.52	3.51	3.50	3.50
Seasonal naive	7.45	7.48	4.38	4.37	5.83	5.83
XGBoost per-segment	3.78	5.26	2.57	3.11	2.38	3.09
XGBoost global (pooled)	3.87	5.65	2.58	3.11	2.36	3.05
Random Forest (multi-output)	4.21	7.24	2.80	3.82	2.59	3.80
XGBoost spatial	3.75	5.54	2.54	3.08	2.32	3.03
GRU no-graph	3.84	5.43	2.60	3.18	2.39	3.07
STGAT	3.83	5.19	2.62	3.13	2.40	3.05
Hybrid: residual	3.72	5.28	2.55	3.08	2.33	2.98
Hybrid: gating	3.76	5.14	2.56	3.07	2.33	2.99
Hybrid: average	3.77	5.25	2.56	3.07	2.34	3.00

Table 31. MAE by time slice (mph).

12.4 Congested vs free-flow

Model	Cong. 15	Cong. 30	Cong. 60	Free 15	Free 30	Free 60
Persistence	7.31	9.63	13.33	2.96	3.43	4.24
Historical average	12.90	12.90	12.90	2.96	2.96	2.96
Seasonal naive	14.79	14.79	14.80	4.19	4.19	4.19
XGBoost per-segment	7.63	9.91	12.15	2.19	2.34	2.49

Model	Cong. 15	Cong. 30	Cong. 60	Free 15	Free 30	Free 60
XGBoost global (pooled)	7.51	10.15	13.03	2.23	2.37	2.48
Random Forest (multi-output)	7.68	10.29	14.05	2.49	2.88	3.42
XGBoost spatial	7.18	9.59	12.58	2.21	2.36	2.48
GRU no-graph	7.27	9.56	11.69	2.27	2.43	2.64
STGAT	7.16	9.19	10.91	2.30	2.44	2.64
Hybrid: residual	6.90	8.86	11.50	2.24	2.42	2.54
Hybrid: gating	7.19	9.28	11.20	2.22	2.38	2.54
Hybrid: average	7.22	9.53	11.72	2.22	2.36	2.50

Table 32. MAE on congested (< 40 mph) and free-flow targets (mph). Congested errors are 3–5 times larger.

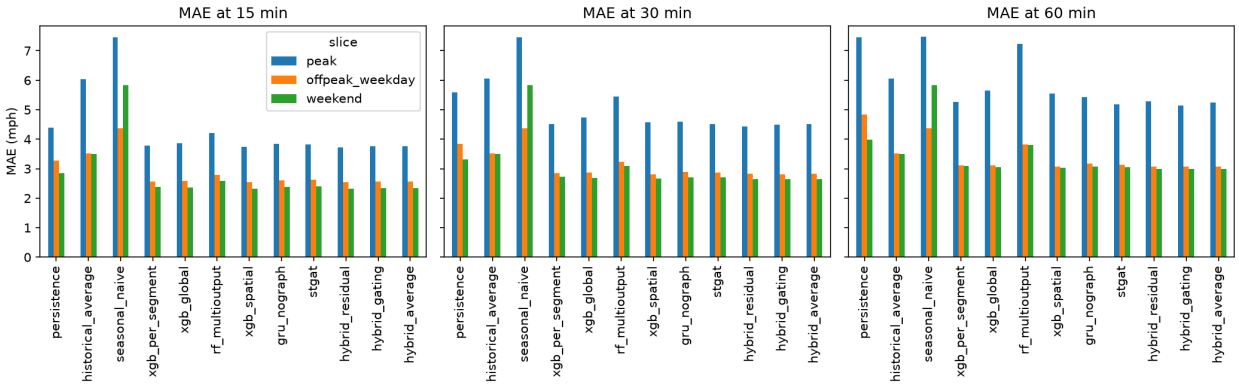


Figure 18. MAE by slice at 15, 30 and 60 minutes.

12.5 Uncertainty: day-block bootstrap

Model	15 min	30 min	60 min
Persistence	3.49 [3.32, 3.66]	4.20 [3.96, 4.43]	5.37 [5.00, 5.73]
Historical average	4.19 [3.94, 4.46]	4.19 [3.94, 4.46]	4.19 [3.94, 4.46]
Seasonal naive	5.50 [5.10, 5.96]	5.50 [5.11, 5.96]	5.50 [5.10, 5.96]
XGBoost per-segment	2.86 [2.71, 3.01]	3.28 [3.09, 3.46]	3.68 [3.47, 3.92]
XGBoost global (pooled)	2.88 [2.72, 3.03]	3.33 [3.13, 3.53]	3.78 [3.54, 4.02]
Random Forest (multi-output)	3.14 [2.97, 3.30]	3.80 [3.58, 4.01]	4.73 [4.45, 5.02]
XGBoost spatial	2.82 [2.67, 2.96]	3.26 [3.07, 3.45]	3.73 [3.50, 3.97]
GRU no-graph	2.89 [2.74, 3.04]	3.30 [3.11, 3.50]	3.77 [3.53, 4.00]
STGAT	2.89 [2.74, 3.03]	3.25 [3.07, 3.44]	3.67 [3.45, 3.90]
Hybrid: residual	2.82 [2.67, 2.96]	3.22 [3.04, 3.40]	3.65 [3.43, 3.88]
Hybrid: gating	2.83 [2.68, 2.98]	3.22 [3.04, 3.41]	3.61 [3.39, 3.84]
Hybrid: average	2.83 [2.68, 2.98]	3.23 [3.05, 3.42]	3.64 [3.42, 3.87]
XGBoost global + daily lag	2.88 [2.72, 3.03]	3.33 [3.13, 3.53]	3.78 [3.54, 4.03]

Table 33. MAE with 95% day-block bootstrap intervals (first seed). Intervals of different models overlap because day difficulty dominates; paired tests (DM, paired bootstrap) are needed to compare models.

12.6 Significance (Diebold–Mariano)

Model A	Model B	Horizon	DM statistic	p-value	Seed pairs	Verdict
XGBoost spatial	XGBoost global (pooled)	15 min	-14.42	0.0000	1	A better
XGBoost spatial	XGBoost global (pooled)	30 min	-11.55	0.0000	1	A better

Model A	Model B	Horizon	DM statistic	p-value	Seed pairs	Verdict
XGBoost spatial	XGBoost global (pooled)	60 min	-6.89	0.0000	1	A better
XGBoost global (pooled)	XGBoost per-segment	15 min	+3.79	0.0002	1	B better
XGBoost global (pooled)	XGBoost per-segment	30 min	+5.34	0.0000	1	B better
XGBoost global (pooled)	XGBoost per-segment	60 min	+6.29	0.0000	1	B better
XGBoost spatial	XGBoost per-segment	15 min	-6.31	0.0000	1	A better
XGBoost spatial	XGBoost per-segment	30 min	-1.68	0.0925	1	no significant difference
XGBoost spatial	XGBoost per-segment	60 min	+2.97	0.0030	1	B better
XGBoost global (pooled)	Persistence	15 min	-50.36	0.0000	1	A better
XGBoost global (pooled)	Persistence	30 min	-34.82	0.0000	1	A better
XGBoost global (pooled)	Persistence	60 min	-22.96	0.0000	1	A better
STGAT	XGBoost global (pooled)	15 min	+4.08	0.0486	3	B better
STGAT	XGBoost global (pooled)	30 min	-4.69	0.0022	3	A better
STGAT	XGBoost global (pooled)	60 min	-5.47	0.0000	3	A better
STGAT	XGBoost spatial	15 min	+17.74	0.0000	3	B better
STGAT	XGBoost spatial	30 min	+2.42	0.6342	3	no significant difference
STGAT	XGBoost spatial	60 min	-3.71	0.0095	3	A better
STGAT	GRU no-graph	15 min	+0.43	0.7194	3	no significant difference
STGAT	GRU no-graph	30 min	-6.21	0.0828	3	no significant difference
STGAT	GRU no-graph	60 min	-6.79	0.0000	3	A better
STGAT	XGBoost per-segment	15 min	+6.49	0.0000	3	B better
STGAT	XGBoost per-segment	30 min	+0.26	0.7975	3	no significant difference
STGAT	XGBoost per-segment	60 min	-0.69	0.9771	3	no significant difference
Hybrid: residual	XGBoost global (pooled)	15 min	-12.27	0.0000	3	A better
Hybrid: residual	XGBoost global (pooled)	30 min	-10.63	0.0000	3	A better
Hybrid: residual	XGBoost global (pooled)	60 min	-9.22	0.0000	3	A better
Hybrid: residual	STGAT	15 min	-21.26	0.0000	3	A better
Hybrid: residual	STGAT	30 min	-9.44	0.0000	3	A better
Hybrid: residual	STGAT	60 min	-1.26	0.4802	3	no significant difference
Hybrid: residual	Hybrid: average	15 min	-7.18	0.0000	3	A better
Hybrid: residual	Hybrid: average	30 min	-5.22	0.0052	3	A better
Hybrid: residual	Hybrid: average	60 min	+1.05	0.2915	3	no significant difference
Hybrid: gating	XGBoost global (pooled)	15 min	-12.60	0.0000	3	A better
Hybrid: gating	XGBoost global (pooled)	30 min	-10.57	0.0000	3	A better
Hybrid: gating	XGBoost global (pooled)	60 min	-9.94	0.0000	3	A better
Hybrid: gating	STGAT	15 min	-25.16	0.0000	3	A better
Hybrid: gating	STGAT	30 min	-13.03	0.0000	3	A better
Hybrid: gating	STGAT	60 min	-11.33	0.0000	3	A better
Hybrid: average	XGBoost global (pooled)	15 min	-10.92	0.0000	3	A better
Hybrid: average	XGBoost global (pooled)	30 min	-11.81	0.0000	3	A better
Hybrid: average	XGBoost global (pooled)	60 min	-10.86	0.0000	3	A better
Hybrid: average	STGAT	15 min	-25.50	0.0000	3	A better
Hybrid: average	STGAT	30 min	-5.98	0.0000	3	A better
Hybrid: average	STGAT	60 min	-3.05	0.0131	3	A better

Table 34. Diebold–Mariano tests (negative statistic: model A has lower loss; p is the largest over seed pairs).

13. Ablation studies

Each ablation changes one setting from the STGAT default. Budget (to suit a laptop): **one seed and 12 epochs** per run, with a re-run of the default under the same budget as the reference (A0). Runs with different history lengths have different first test windows, so all runs are compared on the 6,820 test anchors they share. Differences of a few hundredths of a mph are within seed noise (the 3-seed std of the full STGAT runs is 0.011–0.022 mph).

ID	Change	MAE 15	MAE 30	MAE 60	Δ15	Δ30	Δ60	Time (s)
A0	reference (bidirectional graph, GAT, 4 heads, 2 layers)	2.984	3.407	3.955	+0.000	+0.000	+0.000	509
A1	no neighbours	2.954	3.372	3.820	-0.030	-0.035	-0.135	318
A2	random graph, same edge count	3.027	3.477	4.043	+0.043	+0.069	+0.088	380
A3	$W^\top$ only (upstream senders)	2.995	3.416	3.956	+0.011	+0.009	+0.001	400
A3	W only (downstream senders)	2.998	3.421	3.942	+0.014	+0.014	-0.013	417
A4	GATv2 attention	2.953	3.376	3.934	-0.031	-0.031	-0.021	3828
A4	fixed row-normalised weights	2.957	3.385	3.893	-0.027	-0.023	-0.062	205
A5	1 head	2.967	3.385	3.895	-0.017	-0.022	-0.060	450
A5	2 heads	2.965	3.381	3.919	-0.019	-0.026	-0.036	474
A5	8 heads	2.974	3.403	3.946	-0.010	-0.004	-0.009	536
A6	1 block	2.992	3.437	4.024	+0.008	+0.030	+0.069	257
A6	3 blocks	2.976	3.408	3.963	-0.008	+0.001	+0.008	728
A7	no node embedding	3.132	3.728	4.581	+0.149	+0.321	+0.626	486
A8	no calendar channels	3.077	3.685	4.643	+0.093	+0.278	+0.688	490
A9	zeros kept + mask (DCRNN style)	3.002	3.425	3.964	+0.018	+0.017	+0.009	493
A10	dropout 0	2.947	3.371	3.866	-0.037	-0.036	-0.089	485
A10	weight decay 0	2.929	3.310	3.731	-0.055	-0.097	-0.224	493
A11	history 24 steps	2.978	3.396	3.895	-0.006	-0.011	-0.060	576
A11	history 6 steps	2.975	3.397	3.934	-0.009	-0.010	-0.021	453
A12	distance bias in attention	2.951	3.372	3.899	-0.033	-0.035	-0.056	497

Table 35. Ablation results (Δ vs A0; negative = better than the default).

- **No graph benefit shown under this budget:** the identity graph (A1) is not worse than the road graph — at 60 minutes it is better.
- **Wrong edges hurt:** a random graph with the same number of edges (A2) is worse than both, weak evidence that topology matters.
- **What matters most:** removing the node embedding (A7) or the calendar features (A8) costs 0.6–0.7 mph at 60 minutes.
- **Within noise:** edge direction (A3), attention type (A4), heads (A5), depth (A6), imputation (A9), history length (A11), edge bias (A12).
- **Regularisation (A10):** removing dropout or weight decay helped — with only 12 epochs the model is under-trained, so this says nothing about full training.
- **A13 (hybrid design):** residual vs gating vs average, read from the main table (Section 11).
- The 3-seed, 40-epoch comparison with the GRU (Section 10.7) remains the stronger evidence that the graph helps at 60 minutes.

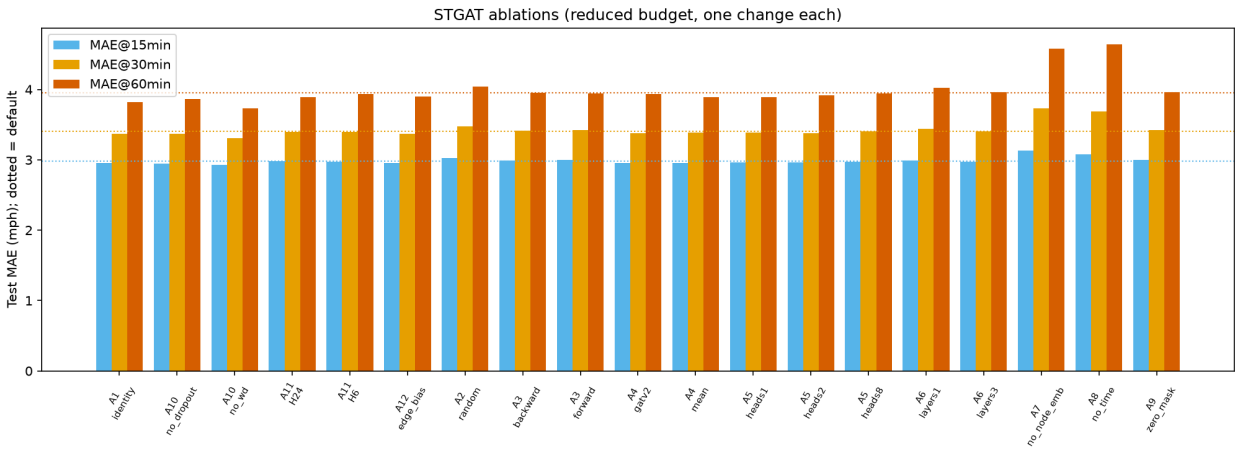


Figure 19. Ablation MAE at 15/30/60 minutes (dotted lines: default).

14. Maps and interpretability

14.1 Forecast maps

The AM-peak map anchor is chosen automatically: the test weekday window whose 15-minute targets fall in 07:00–09:00 with the most congested observed sensors — anchor index 608 with targets 2012-06-06 08:35:00 (+15 min), 2012-06-06 08:50:00 (+30) and 2012-06-06 09:20:00 (+60). Interactive files: `results/maps/forecast_am_peak.html` (truth, STGAT, XGB-global at three horizons, toggleable layers) and `animated_peak_truth.html` / `animated_peak_stgat.html` (time slider over one peak hour). Colours: red < 40 mph, orange 40–55, green > 55, grey missing.

14.2 Where the graph model helps

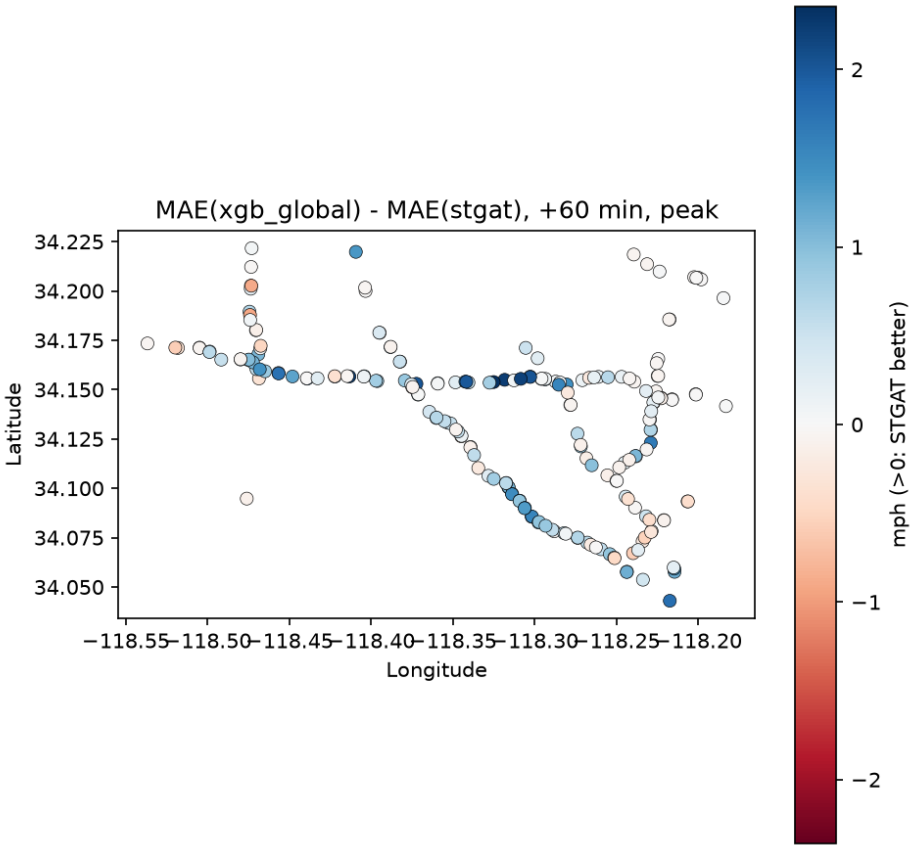


Figure 20. Per-sensor MAE(XGB-global) − MAE(STGAT) at 60 minutes during peaks (blue: STGAT better).

Gains cluster along specific freeway segments; a few sensors are slightly worse with STGAT.

14.3 Space-time diagram

A corridor of 18 sensors is found automatically by following the nearest downstream neighbour with increasing road distance from each start sensor and keeping the longest chain. The day shown (2012-06-21) is the test weekday with the most corridor congestion between 05:00 and 11:00. Neither model anticipates the abrupt breakdown near 10:00 — onset is the hardest event to forecast.

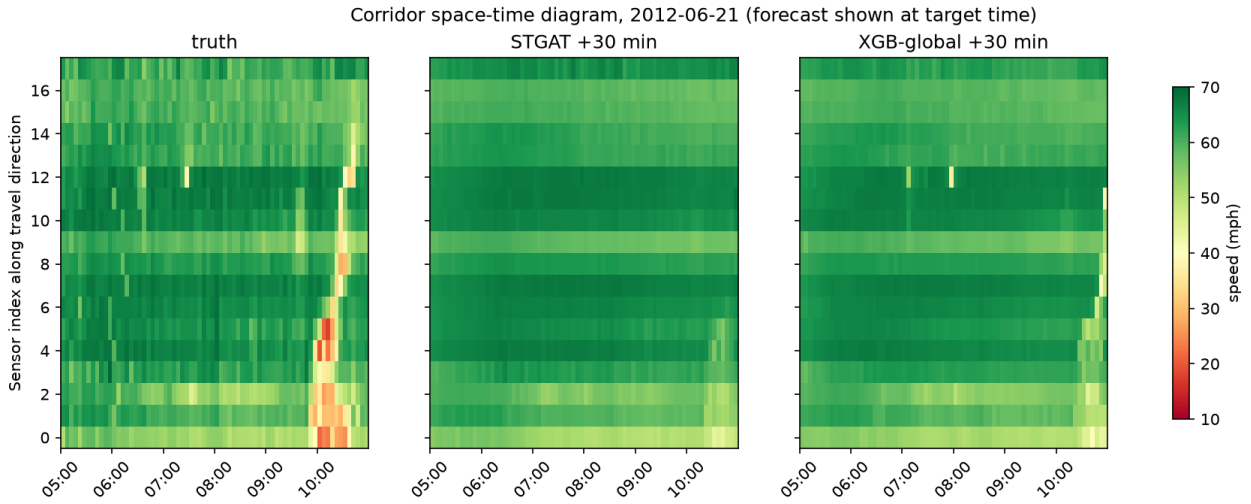


Figure 21. Corridor space-time diagram: truth, STGAT and XGB-global 30-minute forecasts placed at their target time.

14.4 Attention (qualitative)

Period	Windows averaged	Self	Downstream only	Upstream only	Both directions
am peak	72	10.1%	39.3%	39.0%	11.6%
night	136	9.9%	39.0%	39.2%	12.0%

Table 36. Share of last-block attention mass (STGAT seed 0, heads averaged), test weekdays: AM peak = anchors 07:30–08:30, night = 02:00–04:00.

The shares are the same at the AM peak and at night, so we find no peak-specific shift of attention towards upstream or downstream sensors. Attention is a probe, not causal evidence.

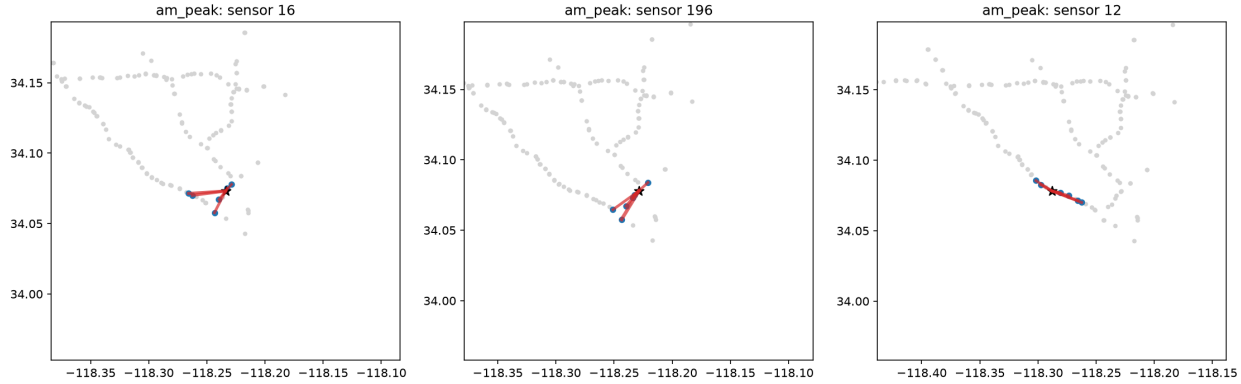


Figure 22. Strongest attention links for the three most congested sensors, AM peak.

15. Simulated ramp-metering experiment

This is an offline simulation on a synthetic network. It demonstrates how forecasts could inform a control policy. No live or real-world traffic control is performed or claimed.

15.1 Why ramp metering

METR-LA sensors are on freeways, which have no traffic signals; the realistic control analogue is on-ramp metering.

15.2 Cell Transmission Model

Element	Setting
Corridor	12 cells × 0.5 mile, 3 lanes, time step 10 s (CFL: 65 mph × 10 s = 0.18 mi ≤ 0.5 mi)
Fundamental diagram	triangular: free-flow 65 mph, capacity 2,000 veh/h/lane, jam density 200 veh/mi/lane, critical density 30.8, backward wave ≈ 11.8 mph
On-ramps	cells 2, 5, 8; maximum ramp discharge 1,800 veh/h
Bottleneck	cell 10 at 75% capacity (lane-drop-like)
Capacity drop	congested cells (lane density > 1.15 × critical) discharge at 88% of capacity
Merge rule	mainline gets 70% priority when the receiving cell cannot take both flows
Upstream boundary	virtual queue for demand that cannot enter

Table 37. Simulator settings (`src/traffic/control/ctm.py`).

Tests check vehicle conservation (in − out − stored = 0 to 1e-6), free flow at low demand, a queue that grows upstream of the bottleneck, ALINEA direction and clipping, and the speed-to-occupancy conversion.

15.3 Demand (synthetic, shaped by METR-LA)

METR-LA has no flow, and speed cannot be inverted to flow in free flow (all sub-critical densities share one speed). Demand is therefore synthetic, shaped by a congestion index computed from the 12 corridor sensors mapped to the cells:

$$\text{ci}(t) = \text{clip}(\ ((65 - \text{mean speed}(t))\ /\ 45)\ /\ \text{ci_ref}\ ,\ 0,\ 1\ ),\ \text{ci_ref} = 0.167 \text{ (95th percentile, training weekdays 05:00–11:00)}$$

$$\text{mainline demand} = 6000 \cdot (0.45 + 0.22 \cdot \text{ci}) \text{ veh/h each ramp} = 250 + 150 \cdot \text{ci veh/h}$$

At ci = 1 total demand is about 16% above the bottleneck capacity. Simulated 04:30–11:00, scored 05:00–11:00, on the five test weekdays with the most AM corridor congestion: 2012-06-21, 2012-06-12, 2012-06-15, 2012-06-07, 2012-06-26.

15.4 Policies

- **No control:** ramps unmetered.
- **Fixed-time:** one rate 06:00–10:00, tuned on the validation weekday 2012-05-29 over the grid [200, 250, 300, 350, 400, 500] veh/h; rates ≥ 350 tied (TTS 4788.8 veh·h, identical to no control), and 400 was selected — non-binding, so it equals no control.
- **ALINEA:** $r(k) = r(k-1) + 70 \cdot (o^* - o(k))$ every 60 s, o = occupancy (%) of the cell downstream of the ramp, o^* = critical occupancy 15.4%, rates clipped to 200–1,800 veh/h.
- **Predictive ALINEA (literal spec):** occupancy = max(measured, occupancy implied by the STGAT +15-minute speed forecast via the congested branch $\rho = w \cdot \rho_\text{jam}/(v + w)$).
- **Look-ahead ALINEA:** a copy of the simulator is rolled 15 minutes ahead from its live state with demand moving to the forecast congestion index; ALINEA uses max(measured, look-ahead occupancy).
- **Oracle versions** of both use the true future speeds (upper bound).

15.5 Results

Policy	TTS (veh·h)	Mainline speed (mph)	Max ramp queue (veh)	Congested cell-minutes	Distance (veh·mi)
no control	2,704.8 ± 426.4	52.4 ± 7.5	0.0 ± 0.0	399.2 ± 275.4	139,350.3 ± 3,088.3
fixed time	2,704.8 ± 426.4	52.4 ± 7.5	0.0 ± 0.0	399.2 ± 275.4	139,350.3 ± 3,088.3
alinea	2,705.5 ± 426.6	57.1 ± 4.4	221.3 ± 111.0	243.3 ± 148.5	139,644.5 ± 3,230.8
predictive alinea	4,783.5 ± 464.0	62.0 ± 1.5	766.0 ± 55.3	78.6 ± 43.2	137,226.1 ± 3,011.2
oracle predictive alinea	3,854.6 ± 363.1	61.3 ± 1.9	607.0 ± 61.5	97.9 ± 62.0	137,879.5 ± 3,170.9
lookahead alinea	2,714.7 ± 434.2	58.2 ± 4.2	243.9 ± 117.4	201.8 ± 142.5	139,656.5 ± 3,221.4
oracle lookahead alinea	2,728.3 ± 435.6	58.1 ± 3.8	248.4 ± 117.0	204.2 ± 128.6	139,647.8 ± 3,240.2

Table 38. Simulation results, mean ± std over the five test weekdays.

- ALINEA cuts congested cell-minutes from ≈ 399 to ≈ 243 and raises mainline speed from ≈ 52 to ≈ 57 mph, but total time spent is unchanged: the delay moves to the ramps.
- Look-ahead ALINEA behaves like ALINEA; its oracle version is no better, so better forecasts would not help this controller here.
- The literal forecast-speed variant over-meters (very long ramp queues, much higher TTS) because real-data forecasts do not respond to the simulator's own state (open loop).
- Conclusion: in this simulation, forecasts gave no measurable benefit.

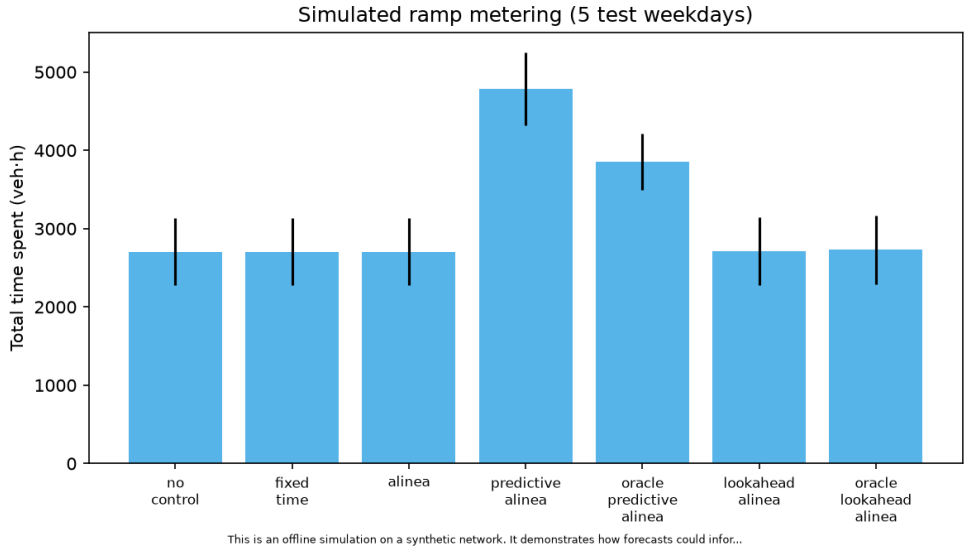


Figure 23. Total time spent by policy (offline simulation).

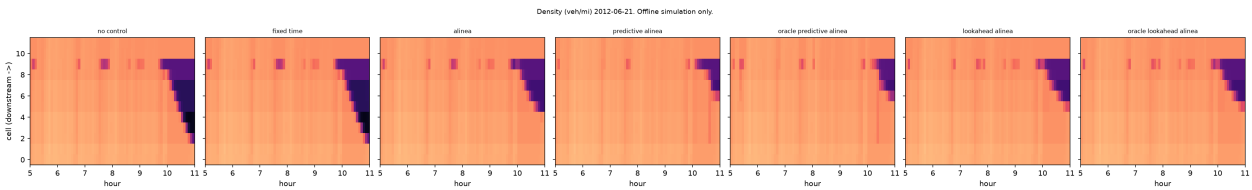


Figure 24. Simulated density (veh/mi) along the corridor per policy, one test day (offline simulation).

16. Engineering and reproducibility

16.1 Repository layout

Path	Content
src/traffic/data	loading, graph, missing data, split, scaling, windows, datasets, pipeline
src/traffic/features	calendar, lag, neighbour features; tabular design matrices
src/traffic/models	baselines, ML models, GAT layers, STGAT, GRU, hybrid, factory
src/traffic/training	masked losses, trainer, Optuna tuning
src/traffic/evaluation	prediction format, metrics, slices, significance, tables, report numbers
src/traffic/viz	EDA plots, maps, horizon plots, attention plots
src/traffic/control	CTM, ALINEA, predictive ALINEA, experiment
scripts/	entry points (download, EDA, train_ml, train_dl, make_oof, train_hybrid, ablations, evaluate, maps, control, report assets) and chain scripts
configs/	base.yaml, ml/, *dl/*, hybrid/*; tuned parameters in ml/*.best.yaml
tests/	unit and end-to-end tests
results/metrics	all CSV/JSON results (tracked)
results/figures, results/maps	figures (tracked) and interactive HTML maps
report/phase1, report/phase2	IEEE reports; every number comes from generated macros in report/tables/numbers.tex
docs/	EDA summary, analyses, theory notes (11), viva preparation, decisions (ADRs), workflow

Table 39. Repository structure.

16.2 Tests

61 tests pass (pytest), and `ruff` and `black` report a clean code base. Key tests: integrity of the raw data; graph rebuild equals the pickle; direction convention on a hand-built chain; no window crosses a split; scaler and imputation statistics depend on training data only; window alignment; features never use the future; tabular row order; masked metrics against hand-computed values; DM and bootstrap behaviour; GAT equals PyG (GATConv, GATv2Conv) and sparse equals dense; the temporal convolution is causal; an identity graph isolates nodes; the hybrid starts exactly at the base forecast; OOF folds never overlap; CTM conserves vehicles; every model runs end to end in debug mode.

16.3 Decision records

ADR	Decision
001	Standalone git repository for the project folder
002	Graph direction convention: row i aggregates from column j
003	Missing inputs set to the training mean in scaled space plus an is_missing channel
004	PyTables fallback for reading metr-la.h5
005	Metric, slice, DM-test and bootstrap conventions
006	Reduced ML budget for a laptop
007	Cheaper STGAT training (spatial steps, epoch fraction)
008	Gating hybrid uses part of validation
009	Spec corrections (CityFlow authors; default bidirectional graph)

Table 40. Architecture decision records (docs/decisions.md).

16.4 Hardware, runtimes and resource limits

- Apple M3 Pro (11 CPU cores, 18 GB RAM); deep models on the integrated GPU through PyTorch MPS.
- Jobs ran one at a time at low priority (nice), with 6 CPU workers for ML and 4 torch threads, to keep the laptop usable.
- Approximate wall-clock: ML tuning + training ≈ 50 min; GRU ≈ 8 min per seed; STGAT ≈ 28 min per seed; OOF + residual hybrids ≈ 80 min; ablations ≈ 4 h 15 min in total (GATv2 alone ≈ 64 min).

16.5 How to reproduce

- Create the environment: `uv venv --python 3.11 .venv`, activate it, `uv pip install -e ".[dev]"`.

- `make data` (download + checks), `make eda`, `make phase1` (baselines and tables).
- `bash scripts/run_ml_chain.sh` (tune and train the four ML models), `bash scripts/run_dl_chain.sh` (GRU and STGAT, 3 seeds).
- `bash scripts/run_phase2_chain.sh` (OOF base forecasts, hybrids, evaluation, maps, control, report assets).
- `bash scripts/run_ablation_chain.sh`, then `python scripts/evaluate_ablations.py`.
- `make report` builds both PDFs (tectonic). Team members joining the repository run `python3 join.py` once.

Reproducibility note. Large intermediate files are not tracked by git. During a clean-up of the repository history, three of them were also removed from the working copy: validation-set predictions (results/predictions_val), the out-of-fold base forecasts (results/hybrid/xgb_base.npz) and the per-run ablation predictions (results/ablations/predictions). All metrics derived from them (main tables, gains, gate weights, ablation table) were already saved and are intact; to re-run the gating hybrid or re-evaluate ablations, regenerate them with the chain scripts above.

17. Limitations and honest findings

17.1 Limitations

- One city and one season (March–June 2012); speed only (no flow, no occupancy).
- A fixed distance-based graph that may miss true connectivity (no adaptive adjacency).
- Reduced budgets: 15 Optuna trials, pooled XGBoost on 25% of rows with a 400-tree cap that was always reached, the forest on 10% of rows.
- STGAT simplified for compute (attention on the last 4 steps, a third of windows per epoch, 40 epochs while still improving).
- Ablations on one seed and 12 epochs.
- Literature-protocol (DCRNN) numbers were not produced; only the strict protocol is reported, so results are not directly comparable to published tables.
- The test period has a higher outage rate than training; MAPE is unstable at low speeds.
- The control study is synthetic, open loop, and has no ramp-queue override or spill-back modelling.
- DCRNN-lite and the SUMO intersection alternative were not built (optional in the specification).

17.2 Results that did not match the hypothesis

- Spatial features help, but not more at longer horizons.
- STGAT equals the no-graph GRU at 15 minutes and beats it only at 60 minutes.
- In the reduced-budget ablations the road graph did not beat the identity graph.
- Attention shares were the same at peak and at night.
- Forecast-informed ramp metering did not reduce total time spent.

17.3 Ethics and authorship

The data are aggregate loop-detector speeds, so privacy risk is low; the main risk is over-claiming real-world control benefits, which this project avoids. Much of the code and documentation was produced with an AI coding assistant and committed as a baseline; team members should follow the course's rules on AI-assisted work and disclose it.

18. Team T042 and contributions

Rank	Member	GitHub	Owns
1	Pranav Singh	Pranav-Singh-Devloper	Deep-learning core: dense/sparse GAT layer, STGAT, GRU no-graph, trainer, DL runs, integration and review
2	Kunal	kunaldevsahu	Phase 1 ML (features, XGBoost, Random Forest, Optuna, SHAP, gap analysis) and the hybrid models
3	Himanshu	HimanshuPal29	Evaluation and statistics framework, baselines, significance tests, simulated control
4	Kanchan	kanchan97-cp	Data pipeline, graph construction, missing-data handling, EDA, ablation runs
5	Vaibhav	VaibhavSingh50	Scaffold/CI, maps and attention plots, README, slides, report assembly

Table 41. Work split, ordered by difficulty and technical depth.

Full module lists, report sections, theory notes and individual viva questions are in `CONTRIBUTIONS.md` and `docs/viva/`. Ownership of every file is recorded in `OWNERS.yaml`.

Appendix A. Glossary

Term	Meaning
Anchor	the last input time step of a window
Horizon h	how far ahead (h × 5 minutes)

Term	Meaning
Masked metric	a metric computed only over observed (non-zero) targets
Strict protocol	chronological split with windows built inside each split
OOF	out-of-fold: predictions made by a model that did not train on those rows
DM test	Diebold–Mariano test of equal predictive accuracy
HAC	heteroskedasticity- and autocorrelation-consistent variance (Newey–West)
GAT / GATv2	graph attention network; v2 has dynamic attention
STGAT	the project's spatio-temporal GAT
TTS	total time spent by all vehicles (veh·h)
CTM	Cell Transmission Model (Daganzo 1994)
ALINEA	feedback ramp-metering law (Papageorgiou et al. 1991)
MPS	Apple's Metal Performance Shaders backend for PyTorch

Table 42. Glossary.

Appendix B. References

1. Veličković et al., Graph Attention Networks, ICLR 2018.
2. Brody, Alon, Yahav, How Attentive are Graph Attention Networks?, ICLR 2022.
3. Kipf and Welling, Semi-Supervised Classification with Graph Convolutional Networks, ICLR 2017.
4. Li, Yu, Shahabi, Liu, Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting, ICLR 2018.
5. Yu, Yin, Zhu, Spatio-Temporal Graph Convolutional Networks, IJCAI 2018.
6. Wu et al., Graph WaveNet for Deep Spatial-Temporal Graph Modeling, IJCAI 2019.
7. Guo et al., Attention Based Spatial-Temporal Graph Convolutional Networks (ASTGCN), AAAI 2019.
8. Zheng et al., GMAN: A Graph Multi-Attention Network for Traffic Prediction, AAAI 2020.
9. Jagadish et al., Big Data and Its Technical Challenges, Communications of the ACM 57(7), 2014.
10. Chen and Guestrin, XGBoost: A Scalable Tree Boosting System, KDD 2016.
11. Breiman, Random Forests, Machine Learning 45(1), 2001.
12. Lundberg and Lee, A Unified Approach to Interpreting Model Predictions, NeurIPS 2017.
13. Diebold and Mariano, Comparing Predictive Accuracy, Journal of Business & Economic Statistics 13(3), 1995.
14. Harvey, Leybourne, Newbold, Testing the Equality of Prediction Mean Squared Errors, International Journal of Forecasting 13(2), 1997.
15. Newey and West, A Simple, Positive Semi-Definite, HAC Covariance Matrix, Econometrica 55(3), 1987.
16. Daganzo, The Cell Transmission Model, Transportation Research Part B 28(4), 1994.
17. Papageorgiou, Hadj-Salem, Blosseville, ALINEA: A Local Feedback Control Law for On-Ramp Metering, Transportation Research Record 1320, 1991.
18. Varaiya, Max Pressure Control of a Network of Signalized Intersections, Transportation Research Part C 36, 2013.
19. Lopez et al., Microscopic Traffic Simulation using SUMO, IEEE ITSC 2018.
20. Zhang et al., CityFlow: A Multi-Agent Reinforcement Learning Environment for Large Scale City Traffic Scenario, WWW 2019.

Reference details were written from the published papers' known metadata; confirm page numbers before formal submission.